Lecture 1

* Computer Organization & Architecture

1. Memory interfacing.
   - Memory hierarchy.
   - Main Memory
   - Cache Memory → 2M.
   - Secondary Storage
     * Mag. disks
     * Mag. Tapes

2M 2. I/O interfacing.
   - I/o transfer
   - data transfer modes.

1. Machine instructions
2. control unit design. } 2M.
3. Addressing modes
4. Data representation. → 2M.

X 5. ALU Data Path.

6. Pipelining. → 1/2M.

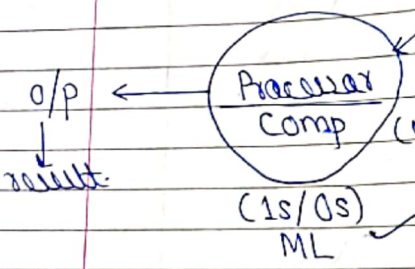
≈ Avg 5-7 M.

Scanned with CamScanner

- 2M from (Cache)
- Data Rep (2M)

* Textbooks: 1. Morris Mano
2. Hamacher → Gate
3. Paul Choudhury

* Basic Terminology



- Prog: Set of instn's
command to perform a task.

* 1 bit represents 0/1.
8 bits rep → 1 byte.
1024 bits → 1 KB (Kilo) 2^{10}
1024 KB → 1 MB (Mega) 2^{20}
1024 MB → 1 GB (Giga) 2^{30}
1024 GB → 1 TB (Tera) 2^{40}

4 bits → 1 nipple

- 1 bit → 0 or 1 → 2 possibilities
2 bits → 00, 01, 10, 11 → 4 poss.
3 bits → 000 – 111 → 8 poss.
.
.
.
K bits → 2^K possibilities

VI

* A digital System represents 986 possibilities. The no. of bits to specify that:

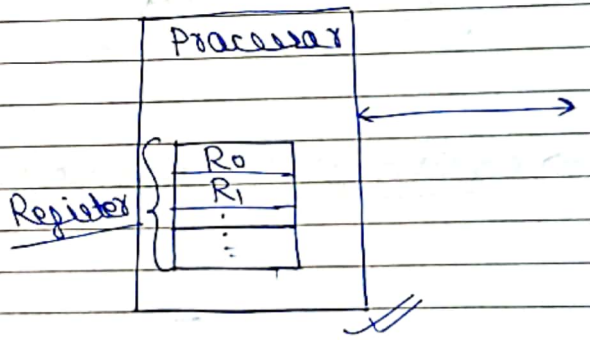
$986 \approx 2^{10}$

10 bits required.

$\Rightarrow \lceil \log_2 986 \rceil = 10$

* Memory Interfacing

[PTO]



Scanned with CamScanner

→ Passive.

- Prog. data resides in sec. storage.
- into MM : Process.

Cache : frq. used by processor.

Processing takes place using ALU/CU.

⇒ Processor rep. just 1 word at a time.

→ Very fast "4 MIPS".

1. Access time ↓ (fast).

→ Accessing from semiconductor mem is fast. (RAM).

→ Cost of Semic. devices ↑ (RAM) ↑.

VI

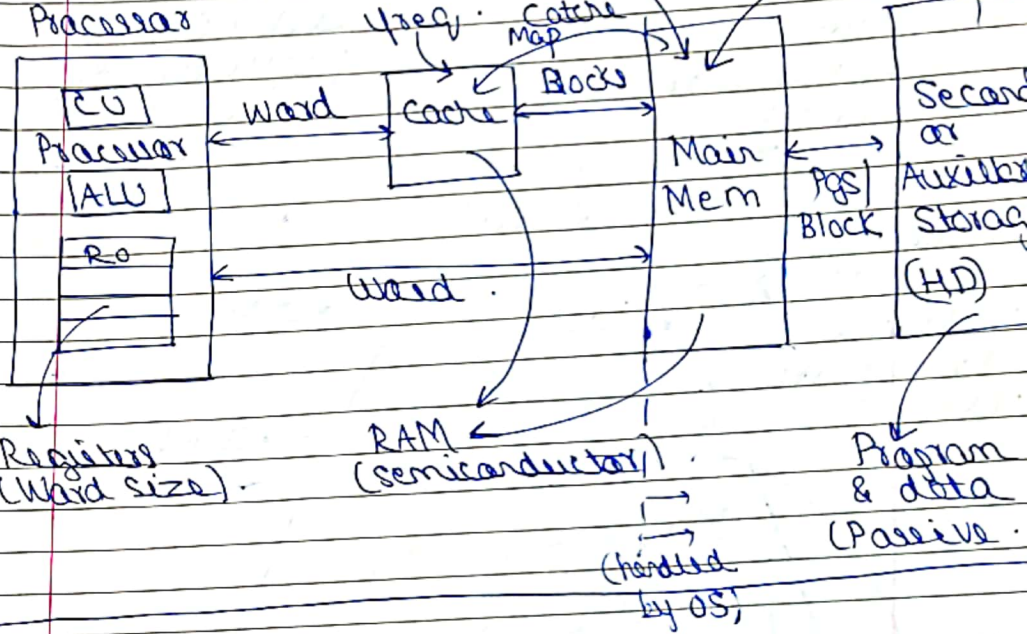
Access Time ↓
Cost ↓
→ ∴ these mory mem. used frq.

- Passive → no lifecycle/no resources.

VI

Very Imp.

(Active) Process → Virtual mem mapp. (OS + MMU)



* Memory Hierarchy

- Purpose of MH is to balance the speed mismatch between fastest processor & the slowest memory at a reasonable cost.

VII

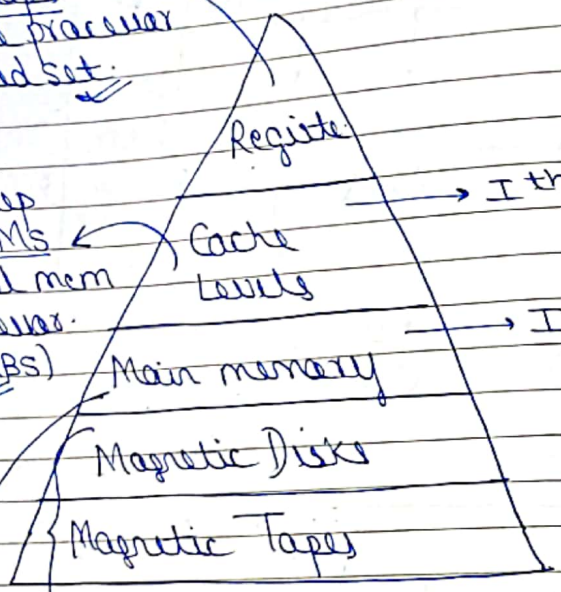
Scanned with CamScanner

Logical Aspects

- Access time (T_i)
- Size (S_i)
- Cost (C_i)
- frequency (f_i)

Physical Aspects

- Flip flops
- wide processor
- limited set.



- made up of SRAMs
- Internal mem to processor.
- (KB — MBS)

- made up of DRAMs
- Internal mem to processor (MB — GBS)

can be accessed directly by processor.

- Magnetising film is used to record info (Iron oxide)
- External storage.
- can be i/o device also.
- (GBS — TBS)

* Access time increases (T_i)
* Size increases (S_i)
* Cost decreases (C_i). Cost of MD < cost of RAM.
* frequency decreases (f_i)

(Processor prefers to take data from top level memories only).

* $i^{th} < (I+1)^{th}$

⇒ In the Mem. Hierarchy:

1. I^{th} level is a subset of $(I+1)^{th}$ level.
2. When Processor refers to I^{th} level memory if it is found → hit otherwise miss / fault.

Scanned with CamScanner

* MAIN MEMORY (RAM)

- collection of rows
- Each row → 1 word
- Each word is accessed by spec. address

2^n rows → n bits required.

- Group of consec words → blocks

[Block Size = 2K]

- Each word → 8 cells (0/1) → byte organised

(1 Word → 8 bits → 1 byte)

byte organised memory

1 word → 16
→ 32
→ 64

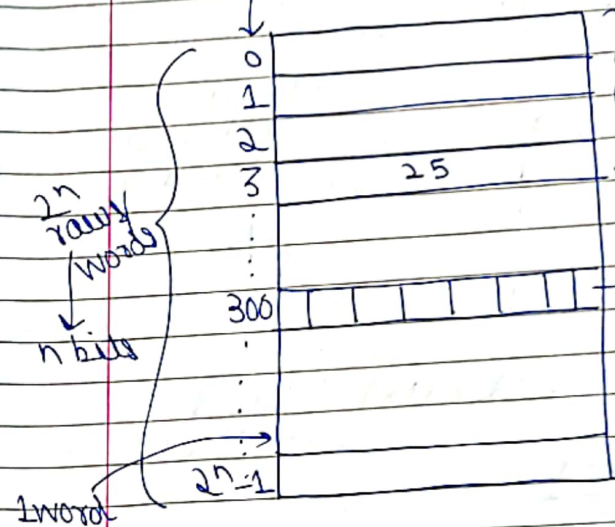
word organised memory

VI

* 1 cell → 0/1 (How?) [Based on structure of 1 cell]

- Static RAM
- Dynamic RAM.

- Organisation is same; differs in how 1 cell is constructed.



1 word → 16, 32, 64, 128
(Word organised memory)

1 word → 8 bits (1 byte)
Byte org memory

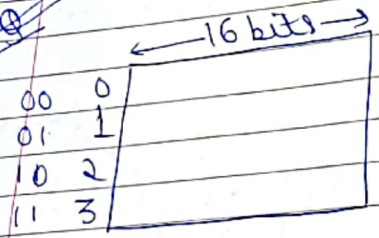
* ii) add = 300 → go to loc.
(Any word is accessed in fixed amount of time)
∴ RAM.

Scanned with CamScanner

→ RAM (: Random Access Memory)

- Once address is specified, any location can be accessed in fixed amount of time. [No Search Time]

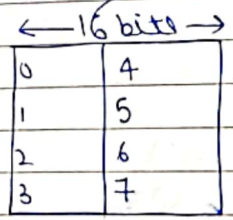
Q



→ Word addressable memory.

No. of rows = 4 ∴ 2 bits) [1 word = 16 bit]

⇒ Byte Add. Memory



∵ 64 bit memory ⇒ 8 words (Byte org.) ∴ 3 bits

No. of rows = 8 ∴ 3 bits.

- Q [Find no. of words].

(Each cell stores 0/1)

SRAM	DRAM
- Static Random Access Memory	Dynamic Random Acc. Memory.
- A group of transistors are used to construct a cell. (approx 6)	· A capacitor is used to construct a cell. (1 capacitor is used)
- Continuous power supply required.	· Capable to store charge.
· No additional h/w required.	· Refreshing circuit is req. (row wise) [Charging done row wise]
- Unit area/cell more in SRAM. (due to 6 transistors)	Unit area/cell is less.
· Small memories req. it. but fast.	Big memories req it (slow).
· Translational delay is less (Transfer is high) · [fast]	Translational delay is more.
· Relatively fast.	· Relatively slow.
· Cache Memories · cost is high. ↑	· Main Memory · Cost is low. ↓

Scanned with CamScanner

* SIZE OF RAM :

- No. of words X No. of bits/word.
  (rows)

eg: 32K * 16

32K rows = 2^{15} rows. ⇒ 15 bit address.

16 bits/row ⇒ 16 data bits.

designed.

major comp. in pc communicate with each other using System bus.

∴ System bus:
- 16 → Data lines.
- 15 → Address lines.
- 7 → Control lines
  - depend on CPU
  - depend on control signals

If 100 CS → 2^7 = 7 bits.

- Capacity = $\frac{32K * 16}{8}$ = 64 KB.
  - in terms of Bytes

Q. How many bits required to address a 4 GB byte organised Main memory.

$2^2 \times 2^{30} = 2^{32}$.

byte organised memory.

no. of rows = no. of bytes =

4 GB

⇒ $2^2 \times 2^{30} = 2^{32}$ rows

⇒ 32 bits req. for address

Q. How many bits required to address a 4 GB memory with a word size of 64 bits.
- Word addressable.

1 Row = 1 word = 64 bits = 8 bytes.

⇒ no. of rows = no. of words (Word Address.)

$\frac{4 \times 2^{30}}{8}$ ⇒ 2^{29} rows.

29 bit address.

Scanned with CamScanner

Q. The possible number of address and data lines to access a RAM size of

$256M \times 32$.

Soln:

- word addressable.

$2^8 \times 2^{20} \Rightarrow 2^{28}$ rows.

- 28 bit address
- 32 data lines

[No. of words = No. of rows]

Q. The possible no. of address and data lines to access 512 MB memory with a word size of 64 bits.

Soln:

64 bits = 8 bytes → word addressable.

$\frac{2^9 \times 2^{20}}{8} = \frac{2^{29}}{2^3} = 2^{26}$ rows.

∴ 26 address lines

64 data lines.

[32 bit data line, acc. it 2 times]

Q. How many RAM chips of size of $32K \times 1$ bit required to construct 512 KB main memory.

Soln:

512 KB → Byte.

$2^9 \times 2^{10}$

$\Rightarrow$ No. of chips $= \frac{512 \times 1K \times 8}{32 \times 1K \times 1}$

$\Rightarrow \frac{2^{9+3}}{2^5} = 2^7 = 128$

Q. Consider a 256 MB DRAM with a word size of 32 bits. The refresh time is 10 nS. What is the time taken to refresh the chip 4 times.

Soln:

$\Rightarrow \frac{2^8 \times 2^{20}}{4 \; 2^2} = \frac{2^{28}}{2^2} = 2^{26} \times 10 \times 10^{-9} \times 2^2$

[4 bytes]

$\Rightarrow 2^{28} \times 10^{-8}$ S

No. of rows $= 2^{26}$ words

$\Rightarrow 2^{26}$ rows

∵ word add.

* 10 nS/row.

$\Rightarrow (2^{28} * 10)$ nS

Scanned with CamScanner

→ [We need only no. of bits in such Qs]

$2^{28} = 10^{K_2}$

$K_2 = 28 \log_{10} 2$

Q. Consider a DRAM with 2^{14} rows. The time for each refresh is 50 nS. What is the percentage of time left used to perform read/write operations if the refresh period is set to 2 ms.

Soln:

Refresh time/chip = $2^{14} * 50$ nS

$\Rightarrow \frac{2^{14} \times 50}{106} = \underline{0.8192}$ mS

$\Rightarrow \underline{0.82}$ mS

Time req. to refresh the chip.

$\Rightarrow 2 - 0.82 = \underline{1.18}$ mS

left for read/write

$\frac{1.18}{2} \times 100 \Rightarrow 59\%$

[refreshment in DRAMs done row wise]

* Memory Hierarchy

- In a 2 level Memory System:

Case 1: (by default)

P → Access Tym of L1

Hit: (Word) T_1, C_1, S_1 → cost/bit,

Fault: L_1

L_2 T_2, C_2, S_2

- if P cant find word in L_1, takes from L_2.

L_1 = catche

L_2 = MM (analogy).

Case 1:

⇒

P

Hit

(Word)

L_1 T_1, C_1, S_1, H_1 → percent. of hit in L_1

Fault CM.

(Word)

L_2 T_2, C_2, S_2

MM

We dont include Search time :: Rare.

Scanned with CamScanner

→ if P wants 80 words, 80 is in L1.

- out of 100 words

80% → L_1 = 80 words
L_2 = 20 words

H_1 (Hit ratio)

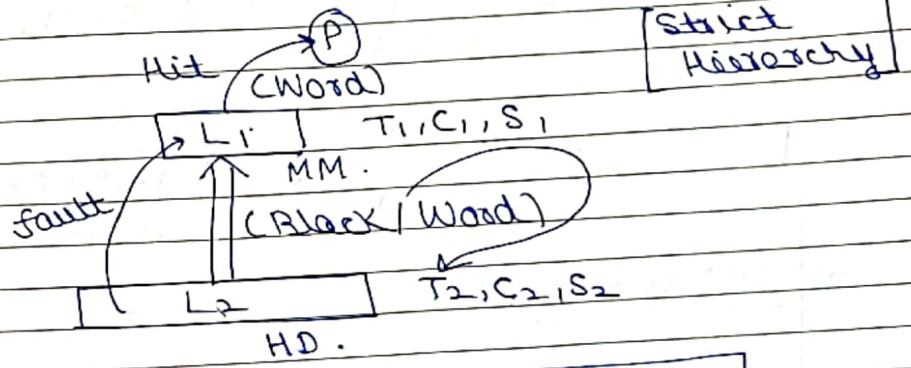
- $T_{avg} = H_1 * T_1 + (1 - H_1) * T_2$

H_1 → Hit rate
$(1 - H_1)$ → Miss Rate
T_1 → Hit time
T_2 → Miss penalty.

- $C_{avg/bit} = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2}$

* Case 2 : (Strict hierarchy)

- P can't directly access SD
∴ SD → MM → P



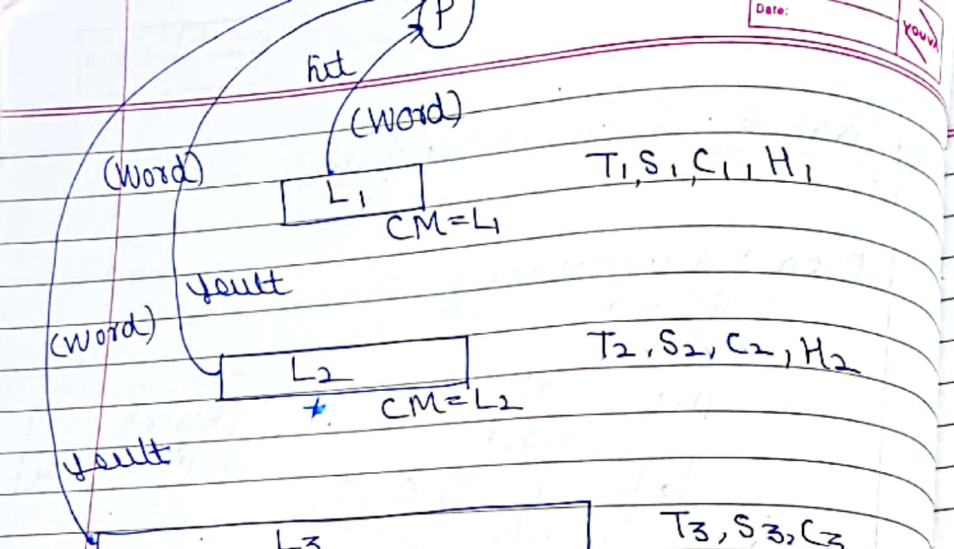
- $T_{avg} = H_1 * T_1 + (1 - H_1)(T_2 + T_1)$

* In a 3 Level Memory System:

Case 1 :

(default case)

Scanned with CamScanner



⇒ 100 words.

80% ⟶ L_1 ⇒ 80 words.

H_1

90% ⟶ L_2 = 18 words.

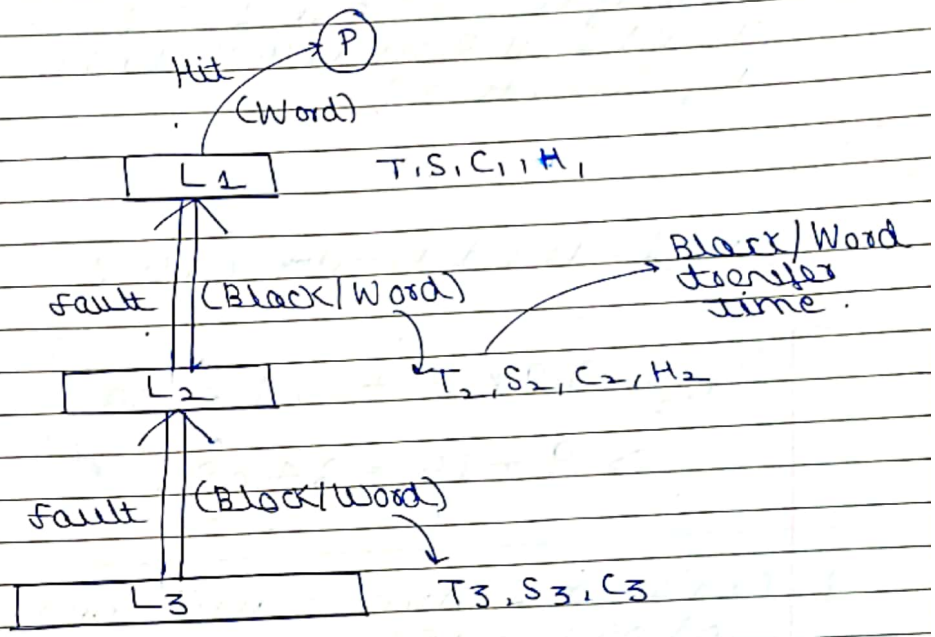
H_2

L_3 = 2 words.

$$\therefore T_{avg} = H_1 * T_1 + (1-H_1) H_2 T_2 + (1-H_1)(1-H_2) T_3$$

$$\therefore C_{avg/bit} = \frac{C_1 S_1 + C_2 S_2 + C_3 S_3}{S_1 + S_2 + S_3}$$

* Case 2 : (Strict hierarchy)



no need to RD.

$$T_{avg} = H_1 T_1 + (1-H_1) H_2 (T_1 + T_2) + (1-H_1)(1-H_2)(T_1 + T_2 + T_3)$$

- Each level fixed access time. VI

→ fixed in time of manufacturing.

Scanned with CamScanner

Q. Consider a 2 level Memory System. The access time of level 1 and level 2 memory is 10 nS and 150 nS. If level 1 Hit Ratio is 90%. What is avg. Access time?

Soln

- case 1: word transfer (2 level)

$\Rightarrow T_{avg} = 0.9 * 10 + 0.1 * 150$

$\Rightarrow 9 + 15 = 24\ nS.$

Q. Consider a System in which cache read miss $\longrightarrow$ 50 nS (T_2) and hit will take 5 nS. When a program is referenced 80% of read request result in hit. What is avg. access time?

Soln.

$T_{avg} = 0.8 * 5 + 0.2 * 50$

$\Rightarrow 4 + 10$

$\Rightarrow 14\ nS.$

$T_2 = 150\ nS$
$T_{avg} = 30\ nS$

Page No.: 23
Date:

Q. In a 2 level Memory System, average access time without level 1 is 150 nS and with L1 is 30 nS.

L_1 (Access time) = 20 nS.

(i) What is the hit ratio?

$T_2 = 150\ nS$ (without L_1)
$T_{avg} = 30\ nS$ (with L_1)
$T_1 = 20\ nS$

$\Rightarrow 30 = x * 20 + (1-x) * 150$
$30 = 20x + 150 - 150x$

$\Rightarrow -120 = -130x$
$x = \frac{12}{13} = 0.92$

$\therefore$ 92.3% $\rightarrow$ Hit ratio.

$H_1 T_1 + (1-H_1) T_2 = 30$
$H \cdot 20 + (1-H)(150) = 30$
$20H + 150 - 150H = 30$
$-130H = -120$

Scanned with CamScanner

(ii) If the hit ratio is made to 100%. What is the access time of L_1 & L_2 memories.

$T_{avg} = H_1 T_1 + (1-H_1) T_2$

$T_{avg} = H_1 = 100\%$

$T_{avg} = 20nS$

$\therefore \begin{cases} T_1 = 20nS \\ T_2 = 150nS \end{cases}$

VI

NOTE: Hit ratio doesn't influence individual memory access times.

↳ fixed during design of chip.

(iii) If avg. access time is increased by 10%, what is the change in hit ratio?

$T_{avg} = 20 + 0.1 \times 2$

$\Rightarrow 30.3 nS$

$\Rightarrow T_{avg} = 33nS$

$\Rightarrow T_1 = 20, T_2 = 150$

$\Rightarrow 33 = 20H_1 + (1-H_1)150$

$\Rightarrow H_1 = \frac{117}{130} \Rightarrow 90\%$

$\therefore$ (2.3% decrease in Hit Ratio)

→ Main from L_2

NOTE: $T_{avg} \uparrow$, $HR \downarrow$ Access Time rem. constant.

(i) At 0.8 Hit in level 1 memory, avg access time is 60nS, level 1 memory is 5 times faster than level 2. If avg. Access time is increased by 20%, what is the percentage range change in hit ratio.

(a) 10% increases
(b) 20% increases
(c) 20% decreases
(d) 10% decreases.

VI

Soln.

$H_1 = 0.8$, $T_{avg} = 60$, $T_2 = 5T_1$.

Scanned with CamScanner

$T_{avg} = H_1 T_1 + (1 - H_1) T_2$

$60 = 0.8 T_1 + 0.2 \cdot 5 T_1$

$0.8 T_1 + T_1$

$60 = 1.8 T_1$

$T_1 = 33.33 nS$; $T_2 = 166.65$

$H_1 = 80\%$, $T_{avg} = 60nS$, $T_2 = 5T_1$.

$\therefore 72 = x(33.33) + (1-x)(166.65)$

$\Rightarrow H_1 = 0.7 \Rightarrow 70\%$

$\therefore$ (d) 10% decrease.

$T_2 = 5T_1$

$H_1 = 0.8$.

Q. Consider a system in which the access time of main memory is 5 times to that of cache. If cache hit rate is 0.8. What is the speed up?

Soln.

$T_2 = 5T_1$. $H_1 = 0.8$.

VI

$$S = \frac{\text{(Time without cache)}}{\text{(Time with cache)}}$$

$$\Rightarrow S = \frac{T_2}{T_{avg}}$$

$$\Rightarrow \frac{T_2}{T_1 H_1 + (1 - H_1) T_2}$$

$$\Rightarrow \frac{Tx}{Tx\left(\frac{H_1}{5}\right) + (1 - H_1) T_2}$$

$$\Rightarrow \frac{1}{\frac{0.8}{5} + 1 - 0.8}$$

$$\Rightarrow \frac{1}{\frac{0.8}{5} + 0.2} \Rightarrow 2.78.$$

$\therefore$ Speed up = 2.78 X

Scanned with CamScanner

$T_{avg} = H_1 T_1 + (1 - H_1) T_2$

$60 = 0.8 T_1 + 0.2 \cdot 5 T_1$

$0.8 T_1 + T_1$

$60 = 1.8 T_1$

$T_1 = 33.33 nS$; $T_2 = 166.65$

$H_1 = 80\%$, $T_{avg} = 60nS$, $T_2 = 5T_1$.

$\therefore 72 = x(33.33) + (1-x)(166.65)$

$\Rightarrow H_1 = 0.7 \Rightarrow 70\%$

$\therefore$ (d) 10% decrease.

$\frac{120 \times 10}{100}$

$\frac{120 \times 60}{100}$

$T_2 = 5T_1$

$H_1 = 0.8$

Page No.: 27

Date:

Consider a system in which the access time of main memory is 5 times to that of cache. If cache hit rate is 0.8. What is the speed up?

Soln.

$T_2 = 5T_1$. $H_1 = 0.8$.

VI

$$S = \frac{\text{(Time without cache)}}{\text{(Time with cache)}}$$

$$\Rightarrow S = \frac{T_2}{T_{avg}}$$

$$\Rightarrow \frac{T_2}{T_1 H_1 + (1 - H_1) T_2}$$

$$\Rightarrow \frac{Tx}{Tx\left(\frac{H_1}{5}\right) + (1 - H_1) Tx}$$

$$\Rightarrow \frac{1}{\frac{0.8}{5} + 1 - 0.8}$$

$$\Rightarrow \frac{1}{\frac{0.8 + 0.2}{5}} \Rightarrow 2.78.$$

$\therefore$ Speed up = 2.78 X

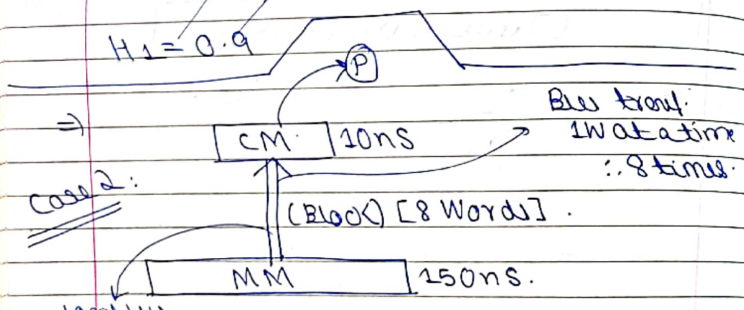
Scanned with CamScanner

Q Consider a 64 word cache and main memory, both are divided into a block of 8 words.
The access time of cache & MM is 10nS and 150nS. (word Access time) when a fault occurs, the associated block is obtained from MM to cache.
What is average access time with 90% cache hit?

Soln.

$T_1 = 10nS$, $T_2 = 150nS$.

$H_1 = 0.9$



Case 2:

$T_1 = 10nS$

$T_B = 8 \times 150 = 1200nS$.

$\Rightarrow T_{avg} = H_1T_1 + (1-H_1)(T_1 + T_B)$.

$\Rightarrow 9 + 121 = 130nS$.

- If bus can transfer (2W) → 4 times

$\Rightarrow T_2 = 4 \times 150 = 600$.

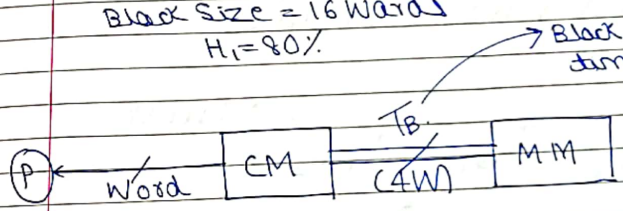
[∵ if bus width = 4W]

$\Rightarrow T_2 = 2 \times 150 = 300nS$.

$T_1 = 10nS$, $T_2 = 150nS$

Block Size = 16 Words

$H_1 = 80\%$



$T_1 = 10nS$, $T_2 = 600nS$ (4 cycles).

$\Rightarrow T_{avg} = H_1T_1 + (1-H_1)(T_1 + T_2)$

$\Rightarrow 0.8 \times 10 + 0.2 \times (610)$

$\Rightarrow 8 + 2 \times 61$

$\Rightarrow 8 + 122$

$\Rightarrow 130nS$.

Q Consider a program:

P0 { 80 → Instruction Fetch.
60 → Operand Fetch.
60 → Write back (write result back)

$H_R = 80\%$, $H_W = 90\%$

$T_1 = 10\,ns$, $T_2 = 50\,ns$

what is avg. access time?

Soln:

$\rightarrow T_{AT}\ 1W.$

$$T_{avg} = \frac{T_{avgR} + T_{avgW}}{2}$$

$$\Rightarrow T_{avg} = \frac{140}{200} \cdot T_{avgR} + \frac{60}{200} \cdot T_{avgW}.$$

$$\Rightarrow \boxed{T_{avg} = f_R \cdot T_{avgR} + f_W \cdot T_{avg.W}}$$

both OP. operation.

$$T_{avgR} = 0.8 \times 10 + 0.2 \times 50$$
$$\Rightarrow 8 + 10 = 18$$

$$T_{avgW} = 0.9 \times 10 + 0.1 \times 50$$
$$\Rightarrow 9 + 5 = 14$$

$$T_{avg} = \frac{14}{20} \times 18 + \frac{6}{20} \times 14$$
$$\Rightarrow 12.6 + 4.2$$
$$\Rightarrow 16.8\ ns$$

$$\therefore \boxed{T_{avg} = 16.8\ ns}$$

Scanned with CamScanner

10/3/19
Sunday

Lecture 2

Q Consider a 2 level cache. The access time of level 1 cache, level 2 cache & Main Memory is 1nS, 10nS & 500 nS. The hit rate of caches is 0.8 & 0.9. What is avg. access time? (Ignore Search time in cache)

Soln:

$$T_{avg} = H_1T_1 + (1-H_1)H_2T_2 + (1-H_1)(1-H_2)T_3$$

$$\Rightarrow T_{avg} = 0.8\times1 + 0.2\times0.9\times10 + 0.2\times0.1\times500$$

$$\Rightarrow 0.8 + 1.8 + 10$$

$$= 12.6$$

$$\therefore T_{avg} = 12.6ns$$

Q Consider a 3 level memory System. The access time of level 1, level 2 and level 3 memories is 100 ns, 150 ns and 500 ns. The L2 & L3 memories are divided into a block of 5 words. When a fault occurs, processor must obtain from L1 memory. The hit ratio of L1 and L2 is 80% and 90%. What is avg. access time?

[Block transfer ∵ divided into blocks]

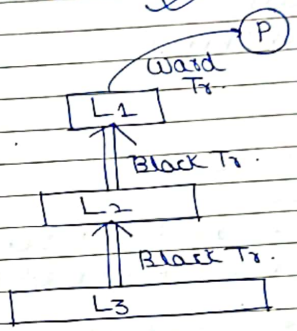
Page No: 33

→ 3 level memory

Case 2:

$T_1 = 100nS$
$T_2 = 750nS$
$T_3 = 2500nS$

VI



$$T_{avg} = H_1T_1 + (1-H_1)H_2(T_1+T_2) + (1-H_1)(1-H_2)(T_1+T_2+T_3)$$

$$\Rightarrow (0.8\times100) + 0.2\times0.9\times750 + 0.2\times0.1\times3350$$

$$T_{avg} \Rightarrow 300ns$$

Scanned with CamScanner

Scanned with CamScanner

In a 2 level memory System;
$L_1 \Rightarrow 32$ Kb
$L_2 \Rightarrow 128$ KB

(cost/bit) $C_1 \Rightarrow x$ units
$C_2 \Rightarrow y$ units
what is avg cost/bit ?

Soln.

$$avgC = \frac{C_1S_1 + C_2S_2}{S_1 + S_2}$$

$$= \frac{x \cdot 32 \cdot 2^{10} + y \cdot 128 \cdot 2^{10}.2^3}{32 \cdot 2^{10} + 128 \cdot 2^{10}.2^3}$$

$$\Rightarrow \frac{2^{10}[32x + 1024y]}{2^{10}[1056 \cdot 2^0]}$$

$$\Rightarrow \frac{32x + 1024y}{1056}$$

$$\Rightarrow \frac{x + 32y}{33}$$

$$\Rightarrow \boxed{C_{avg} = \frac{x}{33} + \left(\frac{32}{33}\right)y}$$

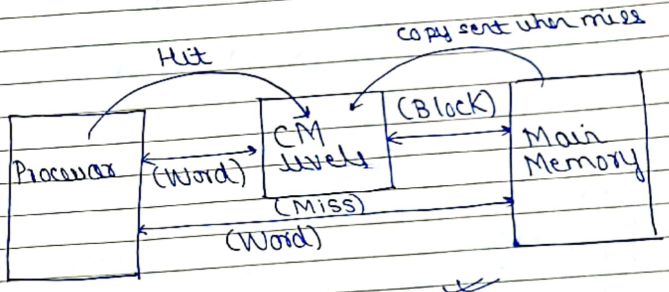
HR↑ . avgT↓

Page No. 35
Date

CACHE MEMORY

(2 M) (distinct)

- small & fast.
- It is a small & the fastest memory.



→ frequently used data.

→ By placing most frequently used data & instructions in cache, average access time can be minimised.

→ Performance of cache depends on hit ratio.

$$HR = \frac{\text{no. of hits}}{\text{total memory refer.}} \times 100$$

HR↑ Perf.↑.

$$\Rightarrow HR = \frac{\text{no. of hits}}{\text{no. of hits} + \text{no. of miss}}$$

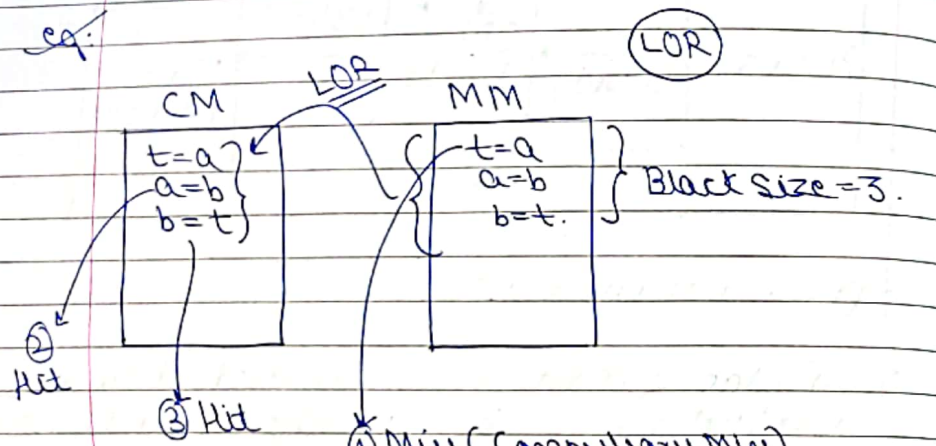
ii) HR = 75 %, MR = 25 %.

* Cache memory works on the principle: locality of reference (LOR)

LOR states that the reference to memory at a given time tends to be confined to within a localised area.

(next ref. from nearby locations only)

eg:



$HR = \frac{2}{3} = 67\%$

- Block Size → 3 → HR = 2/3
  10 → 9/10.

BS ↑, HR ↑

Whenever proc. refers to a word for first time its always miss. (in MM Block)

↳ "Compulsory miss"

- accor to LOR, block is sent to CM.

→ 1st reference to memory block is always miss, called compulsory miss.

→ Hit ratio improves with large size block.

→ The performance of cache (Hit ratio) depends on:

Physical:
1) Cache Size. (Small) [KBs – MBs]
2) Cache Block Size (large)
3) No. of levels of cache (Min – 2 level)
↳ one is inside P. (H)

Logical:
4) Cache Mapping Techniques.
5) Cache Replacement Policies.
6) Cache Updation Policies

VI

Q

- If no. of levels of cache ↑.
  Keeping consistency with data diff.

- CMT: where to place MM block into CM.

Scanned with CamScanner

By default: the
MM must be same.

- Cache is full most of times.
If new block comes, which block is to be replaced.
↳ Cache Replacement Policies.

* CACHE MAPPING TECHNIQUES

- The main memory of the system is having 1024 Blocks of 16 words each.
The cache consist of 64 blocks

• MM address ⇒ 1024 X 16 words
(No. of words/block)

[Word adg. mem] ⇒ $2^{10+4} = 2^{14}$ words.

14 bit address.

* 3 Mapping Techniques:

1. Direct Mapping (DM):

1 page = 64 blocks

⇒ pgs = $\frac{1024}{64}$ = 16 pgs. (4 bits req.)

Mapping: where to place MM block in Cache.

→ Page 0000
Page 0001
⋮
Page 1111.

1. ith block of pg ⟶ ith block of CM.
∴ (fixed position)

V-I

CM

b-0
b-1
b-2
⋮
b-63.

NOTE:

- Every MM Block has fixed location in cache.
i.e. ith block of page must be mapped to ith block of cache
- Cache location of a memory block:

$$CL(M) = M \% n$$

M → Memory Block ref no.
n → No. of cache blocks

∴ n = 64

eg:

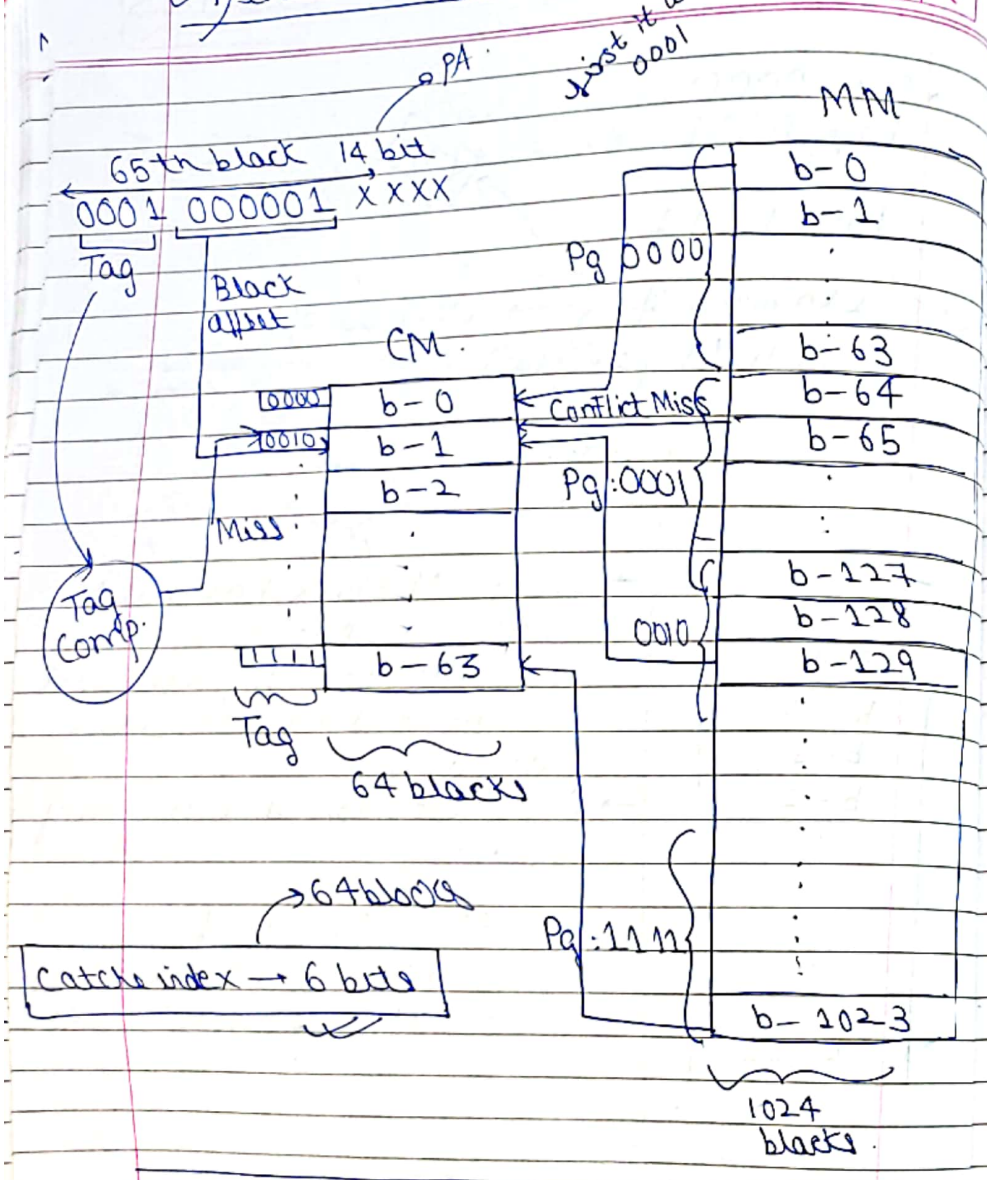
Compulsory Miss:
- CL(0) = 0 % 64 = 0.
- CL(129) = 129 % 64 = 1.
- CL(1023) = 1023 % 64 = 63.

Conflict Miss:
- CL(65) = 65 % 64 = 1.
[Replace Bl: 129 placed in B-1]

↳ : also accessed for first time: comp. miss.

Scanned with CamScanner

Direct Mapping :



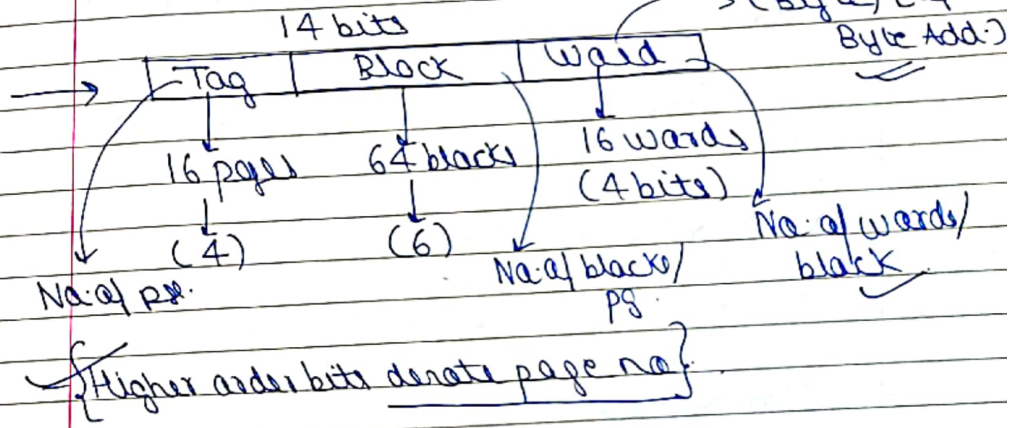
* B-65 conflicted with B-129.

Cache : no rep. address.

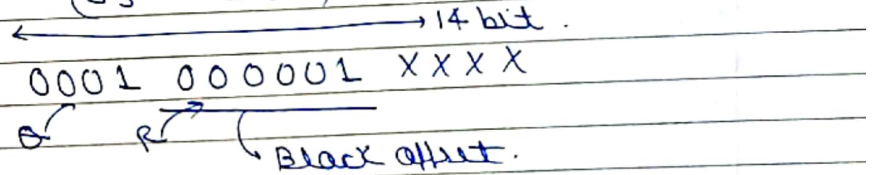
- The replacement is done when a conflict miss occurs. [no replacement strategy in DM]

Pg → Block → word. — 14 bit address.

- The MM address is divided into :



{Higher order bits denote page no.}



⇒ Max Tag comparison = 1. (if Tag pres = Hit. No = miss).

Scanned with CamScanner

→ The higher order bits in the address compared with block offset tag, if a match is found, then hit.
otherwise, miss / fault.
→ Maximum no of tag comparisons = 1.
→ Mapping Process is Simple.
→ Accessing same block from different pages simultaneously is always conflict miss.

[Even though empty blocks, replacement]
HR less, efficiency ↓

→ Hit ratio is very low.

2. ASSOCIATIVE MAPPING (AM)

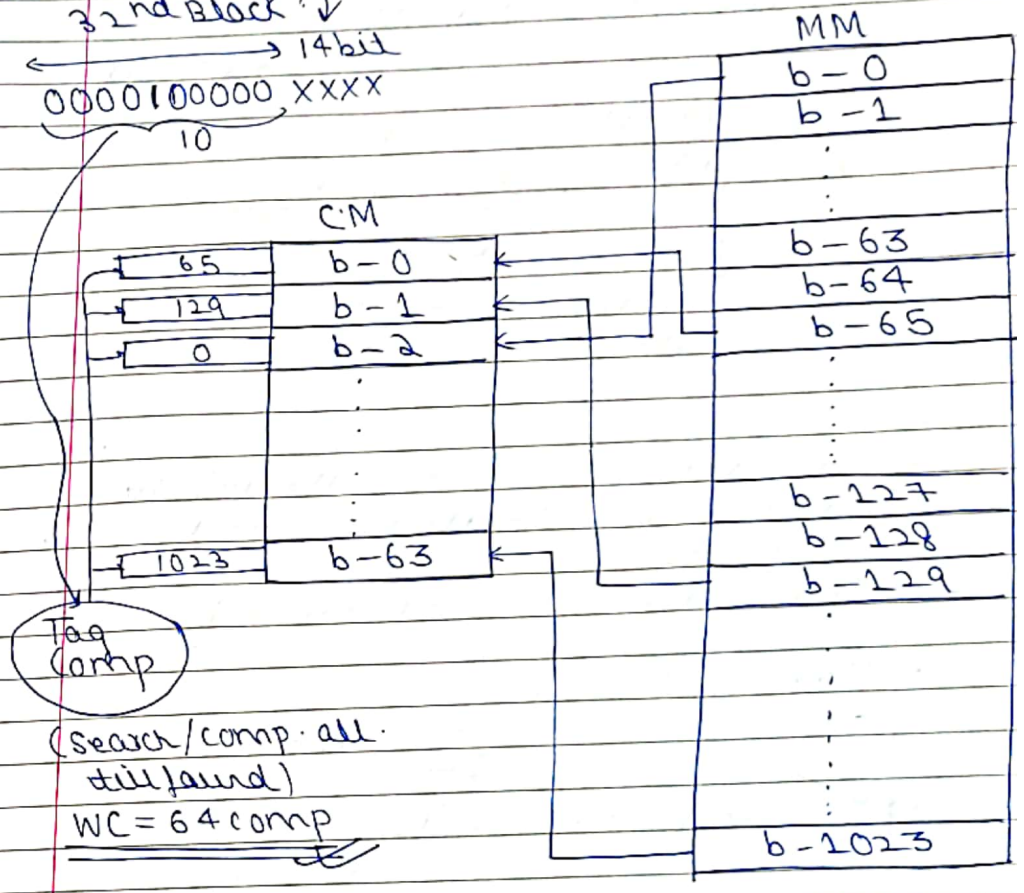
– fully associative.

(Cache index → 64 blocks → 6 bits.)

→ DM.

(any block → AM = 0)

• mapping done randomly.



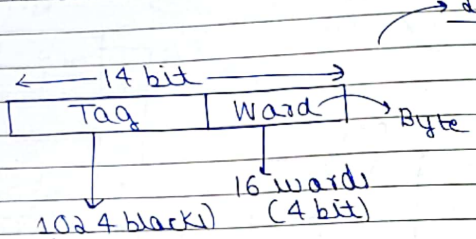
(Search/comp. all till found)
WC = 64 comp

– A MM block is mapped to any location of cache.
– The replacement is done when cache is full (capacity miss)
→ when a new block is req., at that time cache is full.

Scanned with CamScanner

Tag → Block no. (AM)
Tag → Pg no. (DM)

→ MM address is divided into :



→ The higher order bits in the address compared with all the cache tags sequentially.
If a match is formed, then hit.

→ Max no of tag comp = n
i.e. n → 64.

→ No specific cache index. (CI = 0)
∵ start from 0.

→ Hit Ratio is very high
→ Complexity in h/w is very high.
VI → search overhead ↑

SET ASSOCIATIVE MAPPING (SAM)

→ SAM can be K-way SAM.

eg: 2 way SAM
4 way SAM
8 way SAM
⋮

Here, K is no. of blocks in a Set. (Set Size)

* 2 Way-SAM. [2 Blocks in 1 Set]

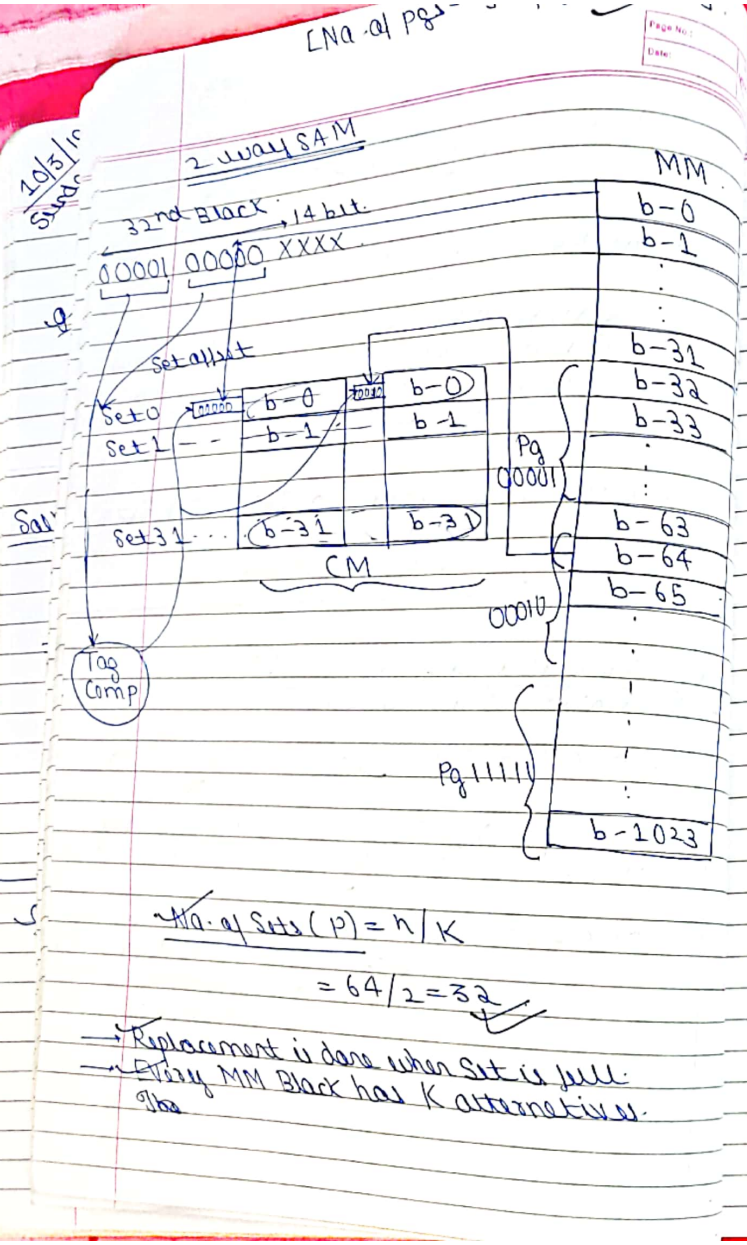
→ ith block of a page is mapped to ith Set.
→ Set location of a memory block:

SL(M) = M % p p = no. of Sets.

SL(0) = 0 % 32 = 0.
SL(64) = 64 % 32 = 0.
} compulsory Miss

SL(32) = 32 % 32 = 0 (conflict miss)
[conf. with 0, 64]
→ replacement
(when Set full)

Scanned with CamScanner



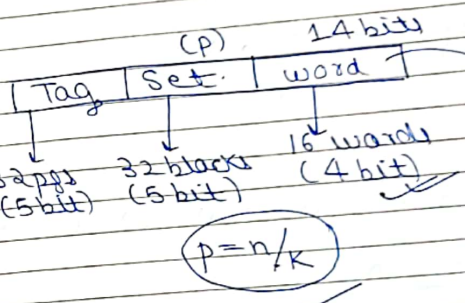
No. of Sets (P) = n/K

= 64/2 = 32

→ Replacement is done when Set is full.
→ Every MM Block has K alternatives.

Tag → Page

- The MM address is divided into:



$p = n/k$

- Max Tag comp = K
- Set index = i th → 32 : 5 bits

- Advantages

(1) HR ↑ (than DM)
(2) Complexity ↓ (than AM)
(3) Simpler Mapping

→ The higher order bits in the address compared with all the blocks of set offset sequentially. If a match is found, then Hit.

Max no. of Tag comp = K (2)

10/3/

- Hit Ratio and complexity in hardware is optimal.

- SAM is of both DM and AM.

** (1 way SAM = DM)

- SAM is implemented with Set of DMs.

→ 2 way SAM can be :

b-0	S-0
b-1	
b-2	S-1
b-3	
b-62	S-31
b-63	

Q Consider a 64 block cache with 16 words/block. The MM of the system is accessed using 16 bit address. How the address is divided / what is the tag size?

4096 Blocks → MM

* DM

16 bit address

Tag	Block	Word

64 (6 bit) 16 (4 bit)

6 bit.

∴ (6, 6, 4) Tag = 6

Catche index : ith place → 64 blocks → 6 bits

* AM

16 bit

Tag	Word

12 bit. 4 bit.

Tag = 12

∴ (12, 4)

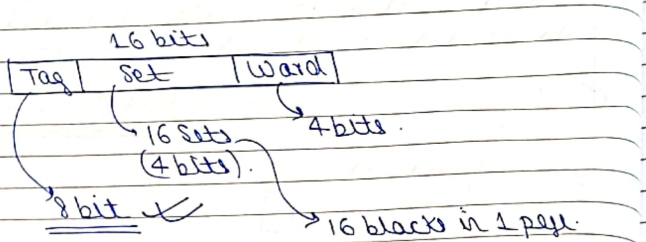
Catche Index : 0

Scanned with CamScanner

Page No: 50
Date:

* 4 Way - SAM

$p = \frac{n}{K} = \frac{64}{4} = 16$

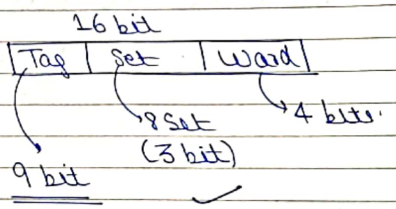


Tag = 8 Div → (8, 4, 4)

⇒ Set Index : ith Set : 16 : 4 bits

* 8 way - SAM

$p = \frac{64}{8} = 8$



Tag = 9 bit. Div → (9, 3, 4)

⇒ Set index : 3 bit

Page No: 51
Date:

As, K ↑ Tag size ↑ complexity ↑

NOTE:

Complexity in h/w (Tag Size) increase with large size Set (K)

VI

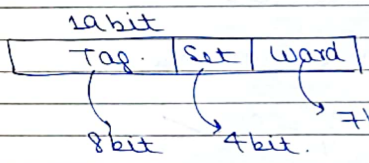
4 way SAM → (better) h/w → 8 way SAM

Q Consider a 4 way SA cache with 16 blocks of 128 words each. The MM of the system consist of 4096 blocks. What is Tag, Set, Ward.

Soln.

4096 × 128 → MM address

⇒ $2^{12} \times 2^7 = 2^{19}$ ∴ 19 bit.



$p = \frac{64}{4} = 16$ sets.

∴ (8, 4, 7)

Scanned with CamScanner

Q. A 2 way Set associative cache has 2C blocks and the MM consist of 2^c blocks. The K+th block of memory is mapped to K%C location.

Soln.

$SL(K) = K \% p$ $\quad p$ = no. of Set.

$p = \frac{n}{K} = \frac{2C}{2} = C$.

$\Rightarrow K \% C$

2 way-SAM (2 Blocks - 1 set).

Range for K th Block.

$SL(K) = (K \% C) + 0$ to $(K \% C) + 1$

Range $\Rightarrow K \% C$ to $K \% C + 1$

** V way SAM with VC blocks.

$p = \frac{n}{K} = \frac{VC}{V} = C$

$SL(K) = K \% p = K \% C$

Range $\Rightarrow (K \% C) + 0$ to $(K \% C) + (V - 1)$

Q. Consider a "8 million word" main memory and 256 block cache. Both are divided into a block of 64 words.

(i) How the address is divided ?
(ii) What is the tag size ?
(iii) Additional memory required for tags ? (Tag memory)
(iv) Max no. of tag comparisons.
(v) Cache capacity. (Total cache size)

for. DM, AM, 4 way SAM, 8 way SAM.

Soln.

1 Million word $\longrightarrow 10^6$ words.

$10^6 = 2^x$

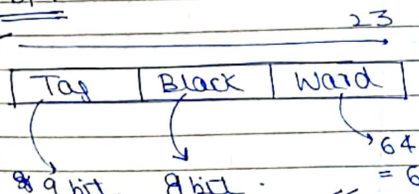
$x = \log_2 10^6 \Rightarrow 2^{20}$.

8 million word $\longrightarrow 2^{23}$ words.

[Word address.] $\longrightarrow$ 23 bit address is used.

* DM

(i)



DM ⇒ 256×9 = 2304 bits
AM ⇒ 256×17 = 4352 bits
SAM ⇒ 256×11 ⇒ 2816
SAM(8) ⇒ 256×12 ⇒ 3072

(iii) DM = 1
AM = 256
SAM (4) = 4 (K)
SAM (8) = 8 (K)

(v) Cache capacity (Total Cache Size)

$$CC = \text{Cache Memory Size} + \text{Cache Directory Size} \quad (\text{Meta Data})$$

Cache Directory = Tag Memory + Valid bit + mem.
Rel bit + distal bit + ...
Alem mem

If Process access a cache block,
Reference bit → 1.
by default: take tag memory.

AM
12 | 6

SAM (4 way)
12 | 6 | 6

SAM (8 way)
12 | 5 | 6

Tag size
(vi) DM → 9 bit
AM → 17
SAM(4) → 11
SAM(8) → 13

(vii) Tag Memory (Every Block → 1 Tag)
Dirty bit
Valid bit R/W
CM
Tag Memory
Ref bit
1 = occupied; I/O → cell

$P = n/K = 256/4 = 64$

Scanned with CamScanner

10/3/

* DM

Case 1:

(By default 1 word = 1 byte)

$$CC = \left[256*64*1\ \text{Byte} + \frac{256\times 9}{8}\right]$$

$$\Rightarrow 16384 +$$

$$CC \Rightarrow (16672)\ \text{Bytes}.$$

Case-2:

Let Size of word is 32 bit. (Word Size)

$$CC = \left[256*64*4 + \frac{256\times 9}{8}\right]$$

$$\Rightarrow 2^8 \times 2^6 \times 2^2 + \frac{28\times 9}{23}$$

$$\Rightarrow 2^{16} + 25\times 9$$

$$\Rightarrow 64K + 25\times 9$$

$$\Rightarrow (64K + 288)\ \text{Byte}$$

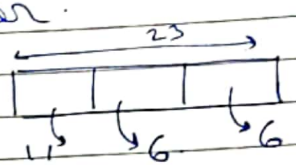
Conclusion: Similarly, for all other mappings.

Page No. 57
Date:

Q. Consider a 4 way SAM with 32 bit word

1 – dirty bit, 1 valid bit, 14 bit, 2 s/w bits. also given.

Cache capacity



(i) tag size : 11 bit.

(ii) tag memory : 2816 bit.

(iii) Cache directory size = 2821 bits

(iv) Cache Capacity =

$$256*64*4 + \frac{2821}{8}$$

VI

$$\Rightarrow 64K + 512$$

$$\Rightarrow 64.5\ KB$$

DM ⟶ Cache Index : no. of block
AM ⟶ Cache Index : 0
SAM(4) ⟶ Cache Index : no. of Sets

$$\Rightarrow (n/4)$$

(n/K), K = no. of blocks/set.

Scanned with CamScanner

Q. Consider a direct mapped cache with 128 lines of 32 bytes each.
The MM of the system consist of 256 MB.
What is Tag, line, Byte.

Soln.

(Byte Addressable)

$2^8 \times 2^{10} \times 2^3 = 2^{28}$ Bytes

28 bit

line 0	B-0	B-1	. . . .	B-31
line 1				
line 127				

28 bit

Tag	line	Byte
28−12	12 bit	5 bit
= 16 bit	(7 bit)	

Q. Consider a byte organised MM of 1MB
(2^20 Bytes) width 16 bytes/block.
A direct mapped cache is of 2^12 lines.
What is tag, line, byte:
in the memory address
(E A 4 3 F)$_{16}$.

Soln:

Soln:

2^20 Bytes → 20 bits

20 bits

Tag	line	Byte
	(12 bit)	(4 bit)
	(4 bit)	

(E A 4 3 F)$_{16}$

1110 0010 0100 0011 1111

Tag = 1110 ⇒ (E)
line = 0010 0100 0011 = (243)
Byte = F.

Scanned with CamScanner

10/3/

Q. Consider a 4 way Set Associative Cache of 32 KB with a block size of 16 bytes. A byte organised MM is having 256MB. What is Tag, Set, Byte.

Soln.

$256\,MB = 2^8 * 2^{20} = 2^{28}$

↳ 28 bits

* ← 28 bits →

Tag	Set	Byte
15 bit	9 bit	4 bit

$= \frac{25 \times 2^{10}}{2^2} = 2^9$

$p = \frac{n}{k} = \frac{2^{11}\ blocks}{4}$

$= 2^9\ sets$

$\therefore$ No. of blocks $= \frac{32 \times 2^{10}}{16} = 2^{11}$ blocks

← 28 →

Tag	Set	Byte
15	9	4

VE

Q. Consider a 64 KB cache with a word size of 32 bits, organised with 8 blocks per set. The MM of the system is having 512 MB. What is Tag, Set, Word? [Block Size = 16 words]

8-way SAM. word add.

Soln.

$512\,MB = 2^9 \times 2^{10} * 2^3 = 2^{22}$.

← 27 →

		Word
		5 bit

$\Rightarrow p = \frac{n}{k} = \frac{2^6 \times 2^{10} \times 2^3}{}$

Word Addressable.

No. of Bytes $= 512 \times 2^{20}$ Bytes

Word Size = 32 bits → 4 bytes.

No. of words $= \frac{512 \times 2^{20}}{4} = 2^{27}$ words.

VE

← 27 →

Tag	Set	Word
16 bit	27 sets (7 bit)	16 words (4 bit)

Scanned with CamScanner

No. of bytes = 64×2^{10} bytes

word size = 32 ⇒ 4 byte

⇒ No. of words = $\frac{64 \times 2^{10}}{4} = 2^{14}$ words

∴ No. of blocks = $\frac{2^{14}}{16} = 2^{10}$.

No. of sets = $\frac{2^{10}}{8} = 2^{7}$ Sets.

word addressable.

GATE 2018 [2 M]

The size of a physical address space of a processor is 2^P bytes. The word length is 2^W bytes. Capacity of cache memory is 2^N Bytes and size of each cache block is 2^M words. For a K-way set associated cache, what is the tag size?

(a) $(P - n - \log_2 K)$

(b) $(P - n + \log_2 K)$

(c) $(P - n - m - w - \log_2 K)$

(d) $(P - n - m - w + \log_2 K)$

Solⁿ:- word addressable

No. of bytes = 2^P

No. of words = 2^{P-W} words

(P − W) bits

| Tag | Set | Word |

2 M words (M bit)

(N − W − M − K) bit.

$K = 2^x$

No. of Bytes = 2^N.

No. of words = 2^{N-W}.

$$\Rightarrow \text{No. of blocks} = \frac{2^{N-W}}{2^M} = \frac{2^{N-W-M}}{K}$$

No. of bits = $2^{N-W-M} \cdot \log_2 K$.

$$\therefore P - W - [N - W - M + \log_2 K]$$

$$P - W - N + W + \log_2 K$$

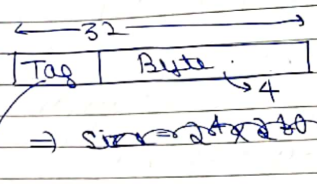
$$\Rightarrow P - N + \log_2 K$$

Scanned with CamScanner

HW GATE-2019

Q A certain processor uses fully associative cache of size 16 KB. The cache block size is 16 bytes. Assume that, the MM is byte addressable and uses 32 bit address. How many bits required for tag & index.

Sol:



$\Rightarrow$ Size = ~~$2^4 \times 2^{10} \times 2^{14}$~~

$\Rightarrow$ ∴ Tag → 28.
index → 0.

Q A 2-way set associated cache has 8 bit tag, while the direct mapped cache uses 6 bits. The delay of tag comparator is $\frac{120}{K}$ ns, where K = tag size.
Let there is a mux of delay 3 ns. What is setting time in both DM and SAM?

Sol:

NOTE: Delay of Tag comparator is called setting time / Hit latency / Hit Delay

VI

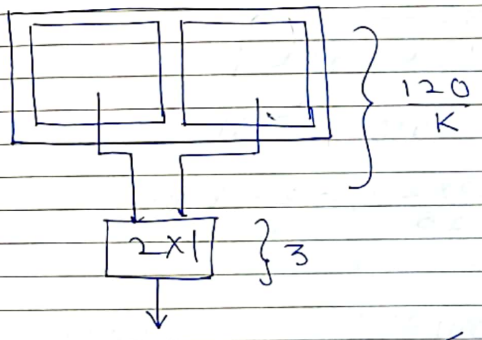
* DM:

$\text{Delay} = \frac{120}{K}$ ← Tag Size

K = 6.

∴ 20 ns.

* 2-way SAM: VI



$\Rightarrow \frac{120}{8} + 3 = 18$ ns.

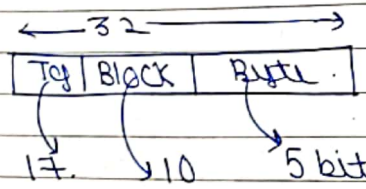
Scanned with CamScanner

Q. Consider 2 cache organizations. One is 32 KB; 2 way Set Associative with 32 Byte Block Size. Another one is of same size, but DM cache.
The size of address is 32 bits in both cases.
Let there is a 2x1 Mux of delay 0.6 ns while a K bit tag comparator has latency of $K/10$ ns.

What is the hit latency in DM, SAM.

Soln:

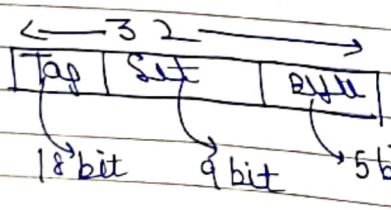
* DM



$\frac{25 \times 210}{25} \Rightarrow 210$ blocks

* 2-way SAM

$\Rightarrow p = n/k = \frac{210}{2}$



$\Rightarrow 17/10 = 1.7$ ns → DM

$\Rightarrow 18/10 = 1.8 + 0.6 \approx 2.4$ ns → 2 way SAM

* CACHE REPLACEMENT POLICIES

- which one to replace.
- mostly cache is full.
- miss penalty ↓

→ replacement strategies are aimed to minimise "miss penalty" for future references.

1. Random
   - No specific criteria

2. FIFO (First in First out)
   - oldest block replaced
   - A block which enters first, leaves first

3. LRU (Least recently used)
   - a block with no references for the longest time. (inactive block)

→ Best

4. LFU (least Frequently used)

A block with lowest references.

NOTE: No replacement Strategy for DM; Required for AM, SAM.

Q) Consider a direct mapped cache with 8 blocks (0-7), the memory block requests are in the order;

3, 5, 2, 8, 0, 63, 9, 16, 20, 17, 25, 18, 30, 24, 2, 63, 5, 82, 17, 24.

Hit Hit Hit

Which of the following memory blocks will not be in the cache at the end of the sequence?

(a) 3
(b) 18
(c) 20
(d) 30.

Soln.

0	8 0 16 24
1	9 17 25 17
2	2 18 2 82
3	3
4	20
5	5
6	30
7	63

CL(M) = M % 8 =

0	8 0 16
1	9 17 25
2	2 18
3	3
4	20
5	5
6	30
7	63

Page No. 69

0	8 0 16 24
1	9 17 25 17
2	2 18 2 82
3	3
4	20
5	5
6	30
7	63

1. Compulsory Miss (3, 5, 2, 8, 63, 30, 9, 20
2. Conflict miss (0, 16, 17, 25, 18,
3. Capacity miss.
   ↳ (24, 2, 82, 17)

∴ (b) 18 not in the cache.

0	8 0 16
1	9 17 25
2	2 18
3	3
4	20
5	5
6	30
7	63

∴ No. of hits = 3
No. of miss = 17.

HR = 3/20, MR = 17/20

No. of compulsory Miss = 8
No. of conflict Miss = 5
No. of capacity Miss = 4

2017

Scanned with CamScanner

Q. Consider a fully associative cache with 8 blocks and the following memory block requests

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 7

* If LRU is used:

The memory block 7 is in 5 cache block?

0	45
1	22
2	25
3	8
4	3
5	7
6	16
7	35

0	45.
1	22.
2	25
3	8
4	3.
5	7
6	16.
7	35.

→ Compulsory Miss: 4, 3, 25, 8, 19, 6, 16, 35

Conflict Miss: NO

[No conflict miss in associated mapping]

capacity miss ⇒ 45, 22, 8, 3, 7

No. of hits = 5
No of miss = 12

HR = 5/17, MR = 12/17.

Q. Consider a 4 way set associative cache with 16 blocks & the following memory block requests:

0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73, 92, 155.

If LRU is used:

Which of the following memory blocks will not be in the cache at the end of the sequence?

(a) 3
(b) 8
(c) 129
(d) 216.

$p = n/k = 16/4 = 4$ sets.

Scanned with CamScanner

0	SL(M) = M % 4 =
1	
2	
3	
4	Compulsary Miss = {0, 255, 1, 4, 3, 8, 133, 159, 216, 63, 73}
5	
6	
7	
8	Conflict Miss = {48, 32, 92, 155}
9	
10	
11	
12	No. of hits = 1
13	No. of miss = 16.
14	HR = 1/17, MR = 16/17
15	

S-0	0, 48
	~~4~~ 32
	8
	~~216~~ 92
S-1	1
	133
	129
	73
S-2	
S-3	~~255~~ 155
	3
	159
	63

→ Capacity Miss

- All misses are applicable

Q. Consider a 2 way set associative cache with 8 blocks & following memory block req.

0, 3, 5, 9, 7, 0, 16, 55

If LRU is used, what memory blocks will be present in the catche at the end of the sequence.

[No. of sets = 8/2 = 4]

0 % 4 = 0
3 % 4 = 3

K = n/p

Set 0	0
	16
Set 1	~~3~~
	5
	9
	7

2 way →

Set 0	0
	16
Set 1	~~3~~ 5
	~~5~~ 9
Set 2	
	~~7~~ 55
Set 3	7

⇒ 0, 5, 7, 9, 16, 55

∴ 3 misses

= Ans.

Scanned with CamScanner

Q Consider a small 2 way Set associative cache with 4 blocks. For choosing the block to be replaced, use LRU. The no. of cache miss for the following block addresses =

8, 12, 0, 12, 8.

M M M M.

2 Sets

8%2 = 0.

S-0		8 0 8
		12
S-1		

∴ 4 Miss.

Q A processor refers to cache memory thousand times in which 150 are resulting faults due to conflict. 100 are of compulsory faults. 100 are capacity faults. What is the hit ratio in:

(a) DM.
(b) AM.

(a) DM = 650/1000 = 0.65 → HR. MR = 35%.

(b) AM = (1000 − [200])/100 = 80%. MR = 20%.

	DM	AM	SAM
Comp	✓	✓	✓
Conflict	✓	X	✓
Capacity	✓	✓	✓

→ NOTE.

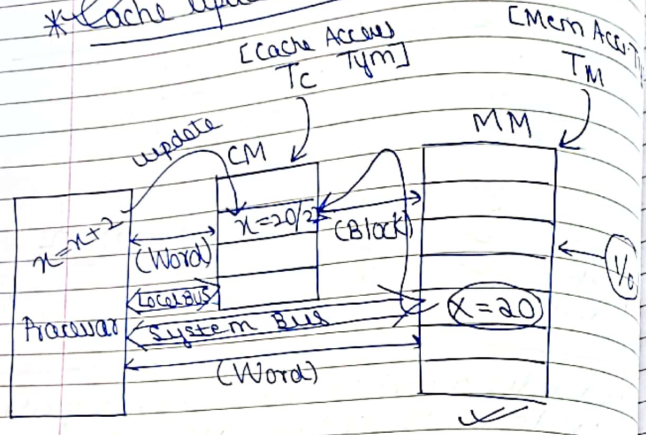
Q Consider a cache has 4 Blocks. No. of hits for the following block addresses:

5, 12, 13, 17, 4, 12, 13, 17, 2, 13, 19, 13, 43, 61, 19

	Ans
(i) FIFO	5
(ii) LRU	6
(iii) Direct Mapping	1
(iv) 2-way SAM with LRU	5

Scanned with CamScanner

* Cache Updation Policies



- when processor modify something in cache, it also has to updated in MM.
  - 2 techniques
    - → write through policy
    - → write back policy.

- T_B: Block Transfer time.

→ The updation in MM is done using:

(i) write through policy:

- simultaneous.
- → Simultaneous updation of a word in both cache and MM.
- → Inconsistency (Cache Coherence) is resolved

- P takes Active prog from [illegible] System Bus.

→ overhead of bus control

- → Always i/o has correct data.
- → The updation of a word takes T_m time. ($T_m \approx T_c$)
- → updation is done with increased bus traffic and overhead. Effective for less updations.

(ii) Write Back Policy → processor involves after replacement.

- → Processor will modify the word only in cache.
- → updation in MM is postponed until the modified block is replaced.
- → Inconsistency can be resolved.
  - CM → 22, MM → 20.
  - not a big issue, ∵ replacement occurs quickly.
- → Effective for more updations. (less overhead)

→ to resolve inconsistency

→ compulsory need to be replaced. D=1 (overwrite in MM).

- when proc. does some updation on Cache.
  - → Unwanted transfer.

Scanned with CamScanner

→ A block which is modified in the cache is called dirty block. Its dirty bit is set to 1.

→ A block is replaced only if its dirty bit is 1.

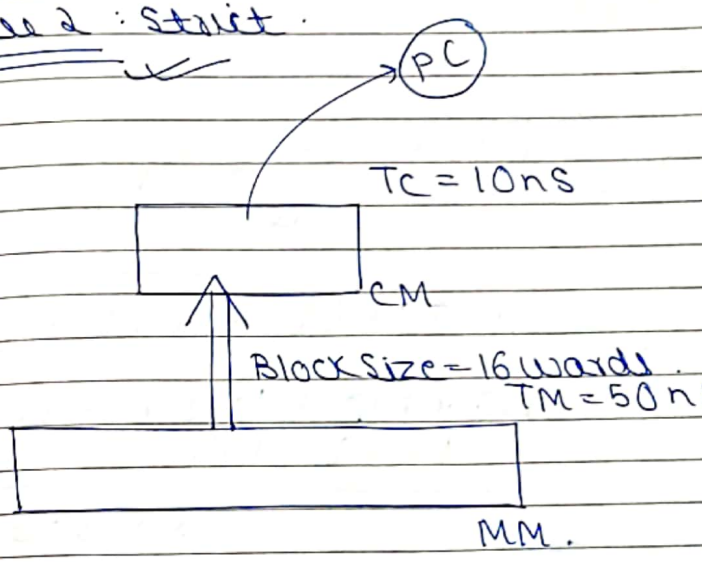
→ NOTE:

(ii) updation in cache means write on cache and vice-versa.

(iii) if no. of dirty blocks is 0, both write through and write back will have same performance.

Q Consider a 64 word cache and MM both are divided into a block of 16 words. The access time of cache and MM is 10nS and 50nS. When a miss occurs, the associated block is obtained from MM to cache for both read & write operations. The hit ratio for read operation is 80% and write operation is 90%. Let there are 40% references for write operations. What is avg. access time and throughput, if a write through updation is used.

- 2 level.
- Case 2 : Strict.



$T_B = 16 \times 50 = 800nS$

$T_M = 50nS$

$H_R = 80\%$, $H_W = 90\%$

$T_{avg} = F_R \cdot T_{avgR} + f_W \cdot T_{avgW}$

$\Rightarrow f_R = 60\%, f_W = 40\%$

$T_{avgR} = H_R * T_C + (1-H_R)(T_B + T_C)$.

$\Rightarrow T_{avgR} = H_1 T_1 + (1-H_1)(T_1 + T_2)$.

$\Rightarrow 0.8 \times 10 + 0.2 \times (800 + 10)$

$\Rightarrow 8 + 162 = 170nS$.

Scanned with CamScanner

T_{avgw} =

case 1:

$$T_{avgR} = H_R T_C + (1-H_R) T_M$$

$$T_{avgw} = H_w T_c + (1-H_w)\cdot(T_B + T_c)$$

$\Rightarrow 0.9 * 10 + 0.1 * (8+10)$

$\Rightarrow 9 + 81 = 90\,ns$

(updation)

→ Write through policy:

$$T_{avgw} = H_w T_m + (1-H_w)(T_B + T_M)$$

- $H_w T_m$ → Hit while write
- T_B → Simult. updation in both CM & MM.
- T_M → Block Transfer from MM to CM

VI

$\Rightarrow T_{avgw} = 0.9 * 50 + 0.1 * (800+50)$

$\Rightarrow 45 + 85 = 130\,ns$

case 1:

$$T_{avgw} = H_w T_M + (1-H_M)\cdot T_M$$

$\Rightarrow H_w T_M + T_M - T_M H_M = T_M$

can straightaway update in T_M.

$T_{avg} = 0.6 * 170 + 0.4 * 135$

$= 102 +$

$T_{avg} \Rightarrow 154\,ns$

* Throughput:

No. Amount of work done/unit time.

- CPU ⇒ no. of instruction/s.
- OS ⇒ no. of process/s.
- Pipelining ⇒ no. of instructn/s.
- Memory ⇒ No. of words stored/s.

VI

f_c = clock cycles/Sec (Hz)

$f_c = \frac{1}{t_c}$ → one clock cycle time

Scanned with CamScanner

$$\text{Throughput} = \frac{1}{T_{avg}}$$

$$\Rightarrow \frac{1}{154 \times 10^{-9}} \Rightarrow \frac{10^9}{154} \text{ words/s}$$

$$\Rightarrow \frac{1000}{154} \text{ million words/s}$$

$$\Rightarrow 6.4 \text{ million words/s}$$

Q. In the above problem, if write back policy is used, what is avg access time and throughput?

→ At one point of time, 30% of cache blocks are replaced

Soln:

$$T_{avg} = f_R * T_{avgR} + f_W * T_{avgW}$$

Case 2:

Boost Tough from MM to cache

$$T_{avgR} = H_R * T_c + (1-H_R)\left[30\%(T_B+T_c) + 30\%(T_B+T_B+T_c)\right]$$

Replace a block C_B

Case 1: (not applicable, some)

writing on cache

Case 2: (write)

writing on cache

$$T_{avgW} = H_W * T_c + (1-H_W)\left[30\%(T_B+T_c) + 30\%(T_B+T_B+T_c)\right]$$

Hit

updation in MM (replace)

writing on cache

Scanned with CamScanner

Lecture 3

* Cache Memory & Arrays

- LOR

Q Consider an array A[100], each element occupies 4 words. A 32 word cache is used and divided into a 8 word blocks. What is the hit ratio for the following statements?

I) for(i=0; i<100; i++)
    A[i] = A[i] + 10; → R & W

→ Memory Structure (Arrays)

① | A(0) | A(1) | A(2) | A(3) | ....
LOR.

② CM

8 word → 2 elem.

A(0)
A(1)
A(2)
A(3)
A(4)
A(5)
A(6)
A(7)

4 blocks

③

	R	W
A(0)	M	H
A(1)	H	H
A(2)	M	H
A(3)	H	H

$$\Rightarrow \boxed{HR = \frac{3}{4} = 75\%}$$

A(0) is compul. miss
⇒ A(1), A(0) is brought to CM by LOR.

eg: if, BS = 3 elements.
HR = 5/6 > 75%

∴ BS ↑ HR ↑ → VI

	R	W
A(0)	M	H
A(1)	H	H
A(2)	H	H

iii) for(i=0; i<100; i++)
{
    x = x + A[i]; → Read.
}

Scanned with CamScanner

① | A(0) | A(1) | A(2) | A(3) | . . . .

② 2 elem. in each block.

③ (only read)

R

A(0) → M
A(1) → H
A(2) → M
A(3) → H

repeat pattern.

② MM

A(0) A(1)
A(2) A(3)

$HR = \frac{1}{2} = 50\%$

∴ R/W pattern affects HR

NO

Q) Consider an Array A[10][10]. Each element occupies 4 words. A 32 word cache is used and divided into 8 Word-Blocks. What is the hit ratio for the following statements?

(i) for(i=0; i<10; i++)
{
for(j=0; j<10; j++)
{
A[i][j] = A[i][j] + 10;
}
}

Soln:

① Array Structure (MS)

eg: $\begin{bmatrix} 3 & 4 & 9 \\ 8 & 2 & 6 \\ 5 & 1 & 7 \end{bmatrix}$ 3×3 → Matrix

⇒ Row Major Order (RMO)

3	4	9	8	2	6	5	1	7
00	01	02	10	11	12	20	21	22

[00 01 02 . . . 0_{n-1} 10 11 12 . . . . 1_{n-1} . . . $(n-1)_{n-1}$]

Scanned with CamScanner

⇒ Column Major Order (CMO)

3	8	5	4	2	1	9	6	7
00	10	20	01	11	21	02	12	22

NOT ✓ [00 10 20 30 ..... 01 11 21 31 .... 0n-1 0 1 n-1 ..... n-1 n-1]

① Memory Structure

② MM

A(0)(0) A(0)(1)	} 8 words / 2 elements.
A(0)(2) A(0)(3)	

③

	R	W
A[0][0] →	M	H
A(0)(1) →	H	H
A(0)(2) →	M	H
A(0)(3) →	H	H

(Both read / write) → rep. pattern.

∴ HR = 3/4 = 75% ✓

(ii) By CMO:

	R	W
A(0)(0) →	M	H
A(0)(1) →	M	H
A(0)(2) →	M	H

CM

A(0)(0) A(1)(0)	
A(0)(1) A(1)(1)	
A(0)(2) A(1)(2)	

HR = 1/2 = 50%

∴ Memory Structure changes HR

(iii)

```
for (j=0; j<10; j++)
    for (i=0; i<10; i++)
        A[j][i] = A[j][i] + 10;
```

R/W

(i) By RMO

	R	W
A(0)(0) →	M	H
A(1)(0) →	M	H
A(2)(0) →	M	H

MM.

A(0)(0) A(0)(1)	
A(1)(0) A(1)(1)	
A(2)(0) A(2)(1)	

HR = 1/2 = 50% ✓

Scanned with CamScanner

(2) Raj CMO

	R	W
A(0)(0) →	M	H
A(1)(0) →	H	H
A(2)(0) →	M	H
A(3)(0) →	H	H

HR ⇒ $\frac{3}{4}$ = 75%.

A(0)(0)
A(1)(0)
A(2)(0)
A(3)(0)

NOTE: [Hault control is accessed also always HRJ

Consider a 32 KB cache with 128 Byte Block Size. Suppose A is a 2D Array of Size 512x512, Each element occupies 8 Bytes. For the following a program segments:

(i) P1:

```
for (i=0; i<512; i++)
    for (j=0; j<512; j++)
        X = X + A[i][j];
```

P1, P2 is executed independently from same initial State.

i, j, X are in registers

→ The no. of code miss of P1 is m1.
and P2 is m2.

Q.i) $M_1 = ?$ (a) 0 (b) 2048 (c) 16384 (d) 262144

(2) Atmaraj go with RMO]
↳ default.

P2:

```
for (i=0; i<512; i++)
    for (j=0; j<512; j++)
        X += A[j][i]
```

0 0
0 1 0
2 0
2 0

Soln:

(1) Atmaraj:

Raj RMO : (default)

$\frac{2^3}{2^3} = 2^4$.

MM: $\frac{128}{8}$ { A(0)(0), A(0)(1) ... A(0)(15) } 16 words allocate.

R	W
A(0)(0)	M H
A(0)(1)	H H
A(0)(2)	M H
A(0)(3)	H H

$MR = 35\%$

R

A(0)(0) → M
A(0)(1) → H
A(0)(2) → H
A(0)(15) → H

pattern repeated

VI

$HR = \frac{15}{16}$. $MR = \frac{1}{16}$

16 illumeb — 1 mili
512×512 — 16,384 mili

(c)

$\frac{M_1}{M_2} =$

(a) 0
(b) 1
(c) 16
(d) $1/16$

$M_1 = 16384$

P2

R	W
A(0)(0)	M M
A(1)(0)	M M
A(2)(0)	M M
⋮	
A(15)(0)	M M

$M_2 = 16 \times 16 \times 16$

(d) $\frac{M_1}{M_2} = \frac{1}{16}$

$M_1 = 1/16$.
$M_2 = 1$.

→ for every column = 1 miss

MM
A(0)(0)
A(0)(1)
⋮
A(0)(15)
A(1)(0)
A(1)(1)
⋮
A(1)(15)
⋮

Page No. Q3 Date

Scanned with CamScanner

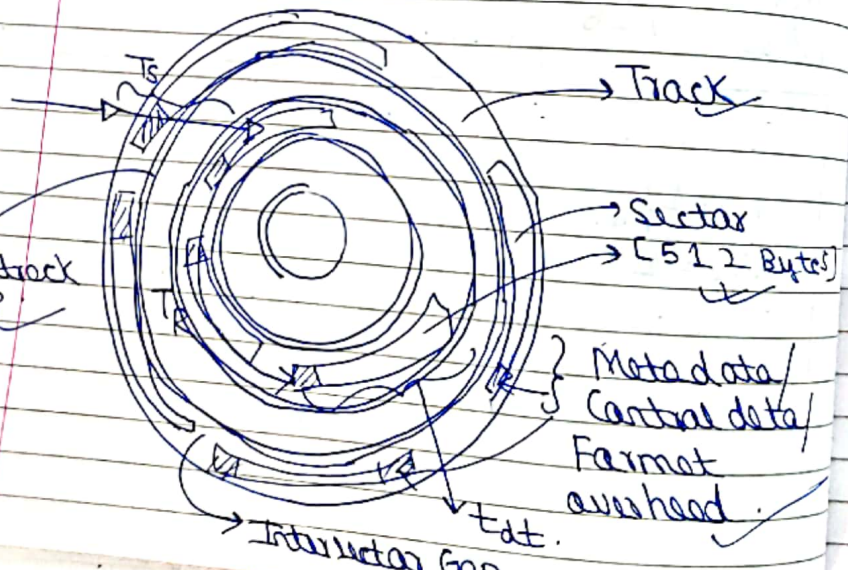
SECONDARY STORAGE

Magnetic Disks

- disks placed one above other. HR

→ A Magnetic disk is a very thin circular metal plate. Data on the disk surface organised as.

- Basic Unit of 1 MD = 1 sector.
- (Metadata/control data/)
  - overhead
  - on beginning of every sector.
- Each track : same no. of Sectors.



- A set of concentric circles called Tracks.
- Each track holds same no. of manageable units, called Sectors.
- Basic unit of Access is a Sector.
- Disk Space without format overhead is formatted disk Space. (effective)
- Storage density on the disk

$$\psi = \text{No. of Bytes / cm}$$

ψ increases from outer to inner track ↑

→ ψ is max at innermost track.

Data transfer rate of the disk:

$$D = \text{No. of Bytes / sec}$$

"RPM"

- D depends on RPM (Rotations per minute).

(Each Rotation covers 1 Track)

→ Seek Time (Ts)

Time to move read/write head to the desired track number.

Scanned with CamScanner

Rotational latency (T_R) [Avg. latency]

The time to move R/W Head to desired sector beginning.

$$t_R = \frac{1}{2} \text{ rotation time}$$

↳ time req. to cover 1 Track

Access Time of Disk

$$t = t_s + t_R$$

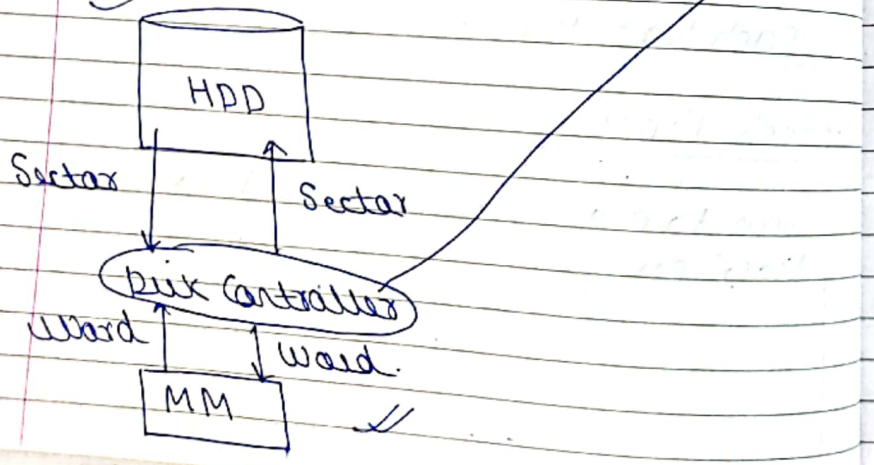
↳ Per Sector / Per Disk Access.

Average Access time of the Disk

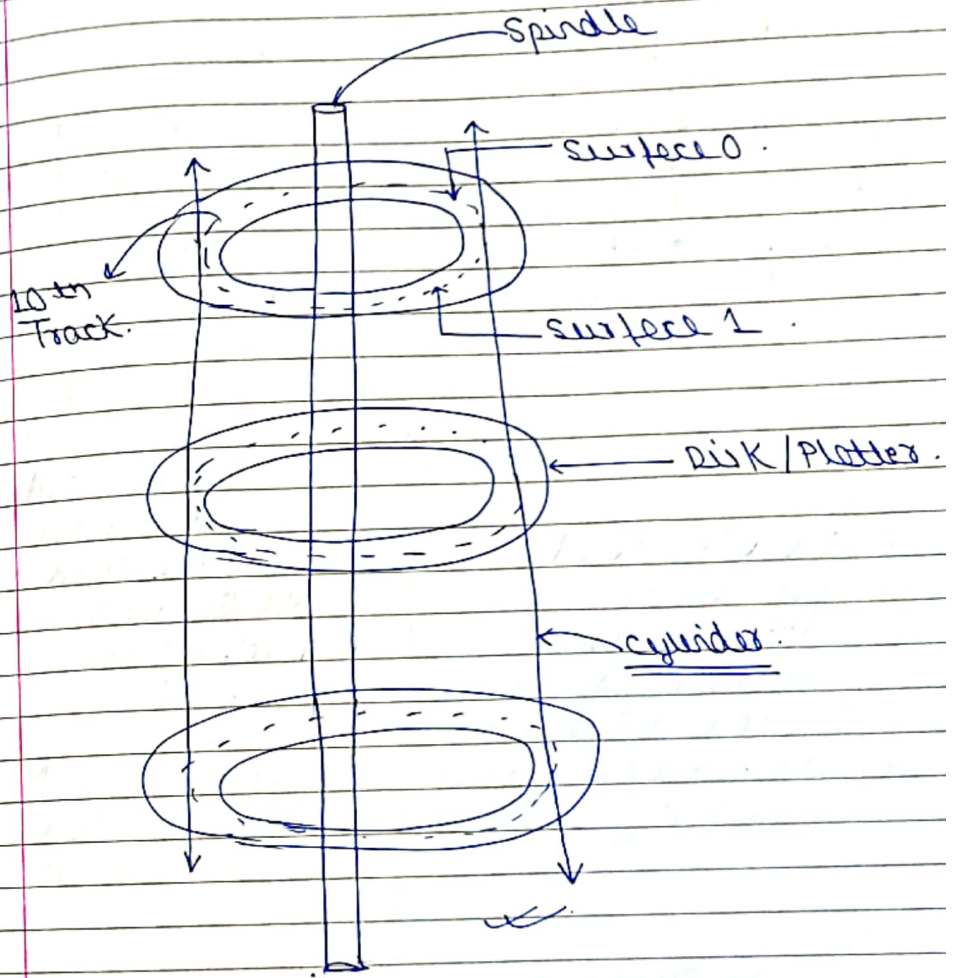
$$t_{avg} = t_s + t_R + t_{dt} + t_{disk\ overhead}$$

(Sector)

↳ per sector.



* Multiple disks can be organised as:



3 characteristics:

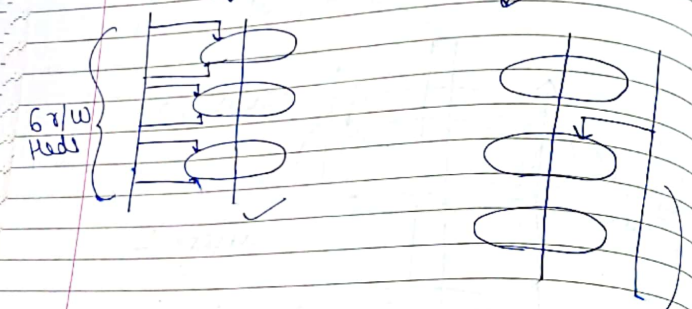
(i) Single Sided / double sided Disks

↳ By default.

50 → 10 surfaces (by default)

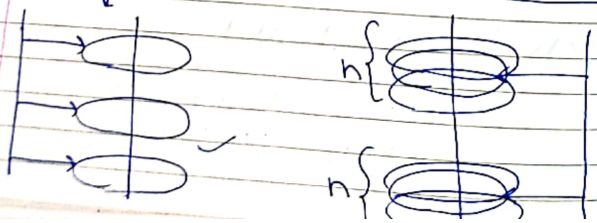
Scanned with CamScanner

(2) Fixed R/W Head / Movable R/W Head Disk



* One R/W Head per surface	* One r/w Head for all surfaces.
* Each Rotation covers n Tracks. [n = no. of Surfaces]	* Each rotation covers 1 track.
* Hardware complexity ↑ ∴ Cost ↑	* Access Time ↑ (Disk → Surface → track → Sector)

(3) Single Platter / Multiple Platter Disks



∴ Multiple Platter Disks mostly used
↳ balances Access Time & Cost

* A vertical Set of all the Tracks at same position in a Disk Pack forms a cylinder.
[No of cylinders = No. of Tracks].

VI

Q1 A disk System has the following specifications

16 Surfaces	→ 4 bit
128 tracks / surface	→ 7 bit
256 Sectors / Tracks	→ 8 bit
512 Bytes / Sector.	→ 9 bit.

} 19

(i) What is the storage capacity of the disk pack?

$C = 24 \times 27 \times 28 \times 29$

$\Rightarrow 2^{28}$ Bytes.

$\Rightarrow$ 256 MB

(ii) How many bits required to address a Sector? on the disk pack?

No. of Sectors = $16 \times 128 \times 256$

$\Rightarrow 2^{4+7+8} = 2^{19}$.

Scanned with CamScanner

∴ 19 bits required.

(iii) In the above disk Pack, format overhead is: 32 bytes / Sector. How much storage is lost due to formatting?

16 x 128 x 256 Sectors (2^{19})

⇒ $(2^{4+7+8}) \times 32$ Bytes

⇒ 2^{24} Bytes → 16 MB

(iv) In the above disk pack, format overhead is 64 bytes / sector. what is the formatted disk Space)

$2^{19} \times 2^6 = 2^{25}$

⇒ 256 MB − 32 MB = 224 MB

⇒ 16 x 128 x 256 x (512 − 64) Bytes

∴ 224 MB

(v) The diameter of the innermost track is 21 cm. What is maximum storage density? (ρ).

VI

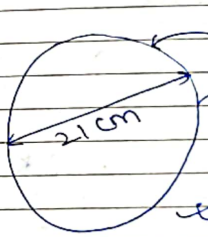
⇒ D = No. of Bytes / cm

Perimeter = $\pi D = \frac{22}{7} \times 21 = 66$ cm.

66 cm ⟶ 1 Track (256 * 512 Bytes)

$\rho_{max} = \frac{1}{66} \times 256 \times 512$ ⇒ 29×28 ⇒

⇒ $\frac{128}{66}$ KB/cm.



(vi) The diameter of the innermost track is 28 cm with a maximum storage density of 2KB/cm. what is the track Storage capacity?

Scanned with CamScanner

Scanned with CamScanner

Perimeter = $\frac{22}{7} \times 28 = 88$ cm.

1 cm = 2 KB
1 track = 176 KB.

∴ Track Capacity = 176 KB

(vii) The disc System has rotational Speed of 3600 rpm. What is the data transfer Rate?

Soln.

$D = \frac{\text{No. of Bits}}{\text{sec}}$

$\Rightarrow 3600 \times 28 \times 29$
$\Rightarrow 217 * 3600$ bytes —— 60 s.

1 rotation = 1 track → movable r/w Head.

3600 rotation —— $60 * 10^3$ msec
1 rotation —— $\frac{60 \times 10^3}{3600}$
$\Rightarrow 16.66$ msec.

16.66 msec → 1 Track (256 × 512)
1000 msec —— x

$D = \frac{1000 \times 256 \times 512}{16.66}$
$\Rightarrow \frac{1000 \times 128}{16.66}$ KBps
$\Rightarrow 7.5$ MBps

VI

(a) fixed r/w Head:

1 rotation = 16 tracks

$\Rightarrow D = 16 \times 7.5$ MBps

Q.2 A Disk System has 19 Surfaces. The storage area on each Surface has an inner diameter of 22 cm and outer of 33 cm. The maximum Storage density is 2000 bits/cm and Spacing among the tracks is 0.25 mm. What is the Storage capacity of the disk pack?

Soln.

$\frac{22}{7} \times 22 = 69.14$ cm.

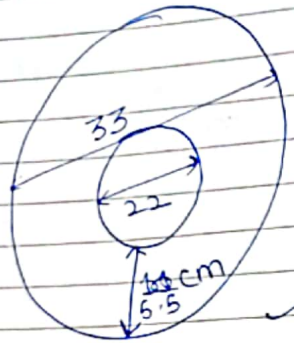
2000 bits —— 1 cm
x —— 69.14 cm
$\Rightarrow$ 138,280 bits → inner track

$\Rightarrow$ 16.879 KB (Track Capacity)

[C = no. of surface * no. of tracks * track capacity.]

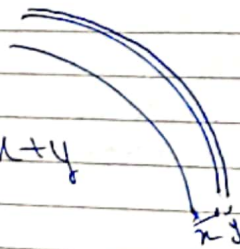
⇒

VI



Width of Storing = from

$\Rightarrow \frac{33}{2} - \frac{22}{2} = 5.5\ cm$



W - Width of storage
x - Width of track
y - Gap.

1 track = x + y

$\therefore$ No. of tracks = $\frac{W}{x+y}$

VI

(If no Gap) = $\frac{W}{x}$

(If only Gaps) = $\frac{W}{y}$. (Gap ↔ Track)

x → not given

No. of tracks = $\frac{5.5}{0.025+} \Rightarrow$ 220 tracks

C = 19 * 16.879 * 220

$\Rightarrow$ 68.9 MB

C = 68.90 MB $\cong$ 68-70 MB

(iii) The Disk System has rotational Speed of 4200 Rpm. What is the Data transfer rate?

4200 rotation ——— 60 S

1 rotation ——— 0.014 S.

$\Rightarrow$ 14.28 ms.

movable.

1 rotn : 1 track $\Rightarrow$ 16.879 KB ——— 14.28 ms

x ——— 1000 ms

$\Rightarrow$

16.879 KB ——— 1 S

0.014

$\therefore D = \frac{1000 \times 16.879}{14.28} = 1.15$ MBps

Fixed: 19 X 1.15 = 21.91 MBps

Scanned with CamScanner

Q5
(2020) A Hard disk has 63 Sectors / Track surfaces 10 Platters. Each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as:

<c, h, s>

where: c = cylinder number
h = surface number [track surface no]
s = Sector number.

Thus, 0th Sector ⟶ addressed as:
<0, 0, 0>,
1st Sector ⟶ <0, 0, 1>
:
: so on.

(i) The address: <400, 16, 29> corresponds to Sector no: _____

(a) 505035
(b) 505036
(c) 505037
(d) 505038

1000 cylinders ⟶ 1000 tracks.

1 Track ⟶ 63 sectors.
1 cylinder ⟶ 20 track Surfaces.
1 cylinder ⟶ 10 x 2 x 63 Sectors

<300, 16, 75>
↓
<300, 17, 12>

<300, 26, 75>
↓
<300, 27, 12>
↓
<301, 7, 12>.

<_____, _____, _____>
- No. of sectors / track
- No. of tracks / cylinder.
- No. of cylinders.

⇒ 504000 +
16 x 63 = 1008.
<400, 16, 29>.

400 x 20 x 63 = 504,000

⇒ (c) 505037.

Scanned with CamScanner

(iii) 1039 sector is ____

(a) <0, 15, 307
(b) <0, 16, 307
(c) <0, 15, 317
(d) <0, 16, 317.

0 + (16 X 63) + 31
⇒ 1039. ✓

(iii) let the size of sector is 512 Bytes.

(a) what is a track capacity?

⇒ 31.5 KB ✓

(b) what is the cylinder capacity?

⇒ 630 KB

(c) what is the storage capacity of the disk pack?

⇒ 1000 X 630 = 615 MB
no. of cylinders.

(d) How many cylinders required to store a file size of 16840 KB?

⇒ 16840 / 630 = 26.7 ≈ 27 cylinders. ✓

* Data Organization on Disk
  - Surface wise
  - Cylinder wise.

Surface wise

Sector address

<Surface no., Track no., Sector no.> ✓

Cylinder wise

Sector address

<Cylinder no., Track Surface no., Sector no.> ✓

The disk system has rotational speed of 3000 rpm. what is avg. access time with a seek time of 11.5 msec?

$t_{avg} = t_s + t_r + t_{dt} + t_{do}$

3000 rotations ——— 60 X 10^3 msec
1 rotation ——— ?

$\Rightarrow \frac{1}{3000} \times 60 \times 10^3 = 20$ msec.

$t_y = \frac{1}{2} \times$ rotn time

$= \frac{1}{2} \times 20 = 10$ ms

$t_{avg} = 11.5 + 10 = 21.5$ msec.

Q Consider a certain moving arm disc storage with 1 r/w Head (movable) has the following specifications:

No. of tracks = 200
track capacity = 62500 bits
Disc Speed = 2400 rpm
Avg latency = p msec
Data Transfer Rate = q bits/sec.

what is p & q? (VI)

Soln

2400 rotations — 60 x 10^3 msec
1 rotn — ?

$x = \frac{60 \times 10^3}{2400} = 25$ msec.

$p = \frac{1}{2}$ rotn time $\Rightarrow$ 12.5 msec

$p = 12.5$ msec

$\Rightarrow$ D = no. of bits / S

$\Rightarrow$

25 msec — 62500 bits
1000 msec — x

$x = \frac{62500 \times 1000}{25}$

$\Rightarrow 2.5 \times 10^6$ bits/S.

$\therefore q = 2.5 \times 10^6$

Q An application loads 100 libraries during startup, where each library takes 1 disc access. The seek time to a random loc^n on the disc is 10 msec and the disc rotates at 6000 rpm. What is the time taken to load all the libraries?

Soln

6000 rat — 60 x 10^3 msec
1 rat^n — ?

$x = \frac{60 \times 10^3}{6 \times 10^3} = 10$ msec

RL = 5 msec. $t_R = 5$ msec.
S $t_S = 10$ msec

$t_1 = t_S + t_R$

$\Rightarrow 10 + 5 = 15$ msec.

$\therefore$ For 100 libraries:

1500 msec = 1.5 sec ✓

Q A disc system has the following specifications:

16 Surfaces
128 tracks/surface
512 sectors/track
1KB/sector
3000 rpm and
1 R/W Head.

$t_S = 10$ msec
$t_{do} = 0.4$ msec.
10MBps

Find out:

What is avg access time?

Soln:

3000 rotn — 60×10^3
1 rotn — x

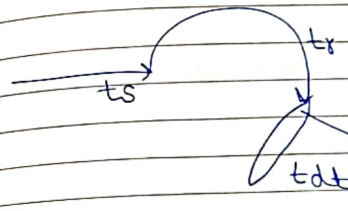
$x = \frac{60 \times 10^3}{3000} = 20$ ms ✓

$t_r = 10$ msec ✓
$t_S = 10$ msec ✓

$T_{avg} = T_S + T_r + T_{do} + T_{dt}$

$\Rightarrow 20 + 0.4 +$

$T_{dt} = \frac{\text{no. of bytes}}{}$



1K = 1000 (time)
1K = 1024 (size)

$\Rightarrow$ 10MB — 1s (10^3 msec)
1KB — x

$x = \frac{2^{10}}{10 \times 2^{20}} \Rightarrow \frac{2^{-10}}{10}$ ✗

$x = \frac{1000}{10} \Rightarrow \frac{1 \times 10^3}{10 \times 10^6} = 0.1$ msec

$t_{avg} = 20 + 0.4 + 0.1$

$t_{avg} \Rightarrow 20.5$ msec

Scanned with CamScanner

Q. Consider a typical disc drive that rotates at 15000 rpm and it has data transfer rate of 50×10^6 bytes/sec. The average seek time is twice to that of rotational delay and the disc controller overhead is 10 times to that of disc data transfer time. What is the time taken to read/write a 512 bytes sector ?

VI

Soln.

$$x = \frac{60 \times 10^3}{15 \times 10^3} = 4 \text{ msec}$$

$\therefore t_R = 2$ msec

$t_s = 4$ msec

$t_{od} = 10\, t_{dt}$ ✗

50×10^6 bytes —— 1 sec.

$t_{od} = 50 \times 10$

15000 rotn's —— 60×10^3 msec.

1 rotn —— ?

$t_R = 2$ msec, $t_s = 4$ msec.

$t_s = 2 t_r$

t_{dt} = 50×10^6 Bytes —— 1×10^3 msec.

512 Bytes —— x

$$t_{dt} = \frac{512 \times 10^3}{50 \times 10^6} = 0.01 \text{ msec}$$

$t_{do} = 0.1$ msec

$t_{avg} = 6.11$ msec

Disk Structures

- constant angular velocity (CAV)
  - fast storage used by HD.
  - 1. track capacity is same
  - 2. density increases from outer to inner track.
  - 3. Access time low.
- Constant Linear Velocity (CLV)
  - Size ↑
  - 1. density is constant
  - 2. Track capacity varies (↑) [increases] from inner to outer.
  - 3. Access time ↑.

Scanned with CamScanner

CLV → optical disc / CDs.

sequential.

CAV

block capacity = 10KB

No of tracks = 10

C = 100KB

CLV

No. of tracks = 10.

$10 + 30 + 30 + \cdots 100$

$\Rightarrow 10\{1 + 3 + \cdots 10\}$.

$\Rightarrow$ 550 KB

$\therefore$ Size (CLV) > Size (CAV)

* MAGNETIC TAPES

- It is a sequentially processed storage
- Data on the tape organised as :

1 char → 8 bits → 8 tracks

1 row → 16 tracks.

tape moves w.r.t CU.

(Density → constant)

- Data should be read place when the tape is moving with a uniform velocity (with fixed R/W heads.
- Storage density on the tape (ϕ)

ϕ = No. of bytes / inch ——— (1)

DT

φ = No. of Bytes/inch-track → (2)

* Data transfer Rate of the tape depends on:

(i) Speed/velocity of the tape
(ii) Constant storage space/density.

Max. data transfer Rate: (D)

$t_{dt} = V \cdot N \cdot \varphi$ (2)

inch's → No. of bytes/inch

* Capacity of the Tape:

$C = L * N * \varphi$ (2) $\quad \epsilon = L * \varphi$ (1)

⇒ inches * No. of bytes/inch

* Utilization factor of the Tape:

$$U = \frac{B_L}{B_L + G_L}$$

* Effective data transfer rate (D_{eff})

$D_{eff} = U * D$

$C_{eff} = U * C$

→ effective capacity.

Q) Calculate the Utilization factor of the tape where Gap length is 0.5 inch, Storage density is of 3000 bytes/inch and has Block storage capacity of 6KB.

Soln:

$$U = \frac{B_L}{B_L + G_L}$$

1 inch ⟶ 3000 bytes
x ⟶ 6KB

$x = \frac{6000}{3000} = 2$ inches

$$B_L = \frac{B_c}{\varphi} = \frac{\text{Bytes}}{\text{Bytes/inch}}$$

$\therefore U = \frac{2}{0.5 + 2} = 0.8$

$U = 0.8$ → 80% tape utilized.

Scanned with CamScanner

Q. Suppose data on the tape is organized into blocks. Each consist of 32 K bits and a gap of 0.4 inch separates the blocks from each other. The maximum storage density is 6250 bits/inch and it has 2400 feet. How many bytes are stored on a tape of 2400 feet?

Soln:

VII

6250 bits ——— 1 inch
32K bits ——— x (B_L)

Or Capacity (C) = No. of Blocks * B_C

- No. of Blocks = $\frac{L}{B_L + G_L}$

$\Rightarrow \frac{2400 \times 12}{5.12 + 0.4} = 5217$

$B_L = \frac{B_C}{\rho} = \frac{32 \times 10^3}{6250} = 5.12$ inch.

Capacity = 5217 x 32K bits.

C ⇒ 20.87 MB

Q. Consider a tape moving with a velocity of 50 inches/S and it has storage density of 110 Kb/inch-track using 8 parallel tracks. Max Data transfer rate is ____?

Soln:

$T_{dt} = V \cdot N \cdot \rho$

$\Rightarrow 50 * 110\,Kb * 8$

$\Rightarrow 5500$ KBps

$\Rightarrow 5.5$ MBps

Q. Consider a tape having 8 m length, moving with a velocity of 50 cm/S and it has storage density of 110 Kb/track-cm using 8 parallel tracks.

(i) what is the capacity of the tape.

Soln. $C = L * N * \rho$

$= 8 * 8 * \frac{110}{cm}$

$\Rightarrow 800\,cm * 8 * 110\,Kb$

$\Rightarrow 88$ MB

C = 88 MB

Scanned with CamScanner

(iii) Let the length of a record is 2.4 cm and a gap of 0.6 cm seperates each other. What is effective data transfer rate?

Soln:

$D_{eff} = u * D$

$D = V * N * \phi$

$= 50 \times 8 \times 110$ Kbps

$\Rightarrow$ 5500 KBps.

$u = \frac{2.4}{2.4 + 0.6} = 0.8$

$D_{eff} = 0.8 \times 5500$

$= 4.4$ MBps

$\therefore$ **$D_{eff} = 4.4$ MBps**

(iii) What is block Storage capacity?

$B_C = B_L \cdot \phi$ = ①

$B_C = B_L \times N \times \phi$ ②

$\Rightarrow 2.4 \times 8 \times 110$ Kb

$B_C \Rightarrow 264$ KB

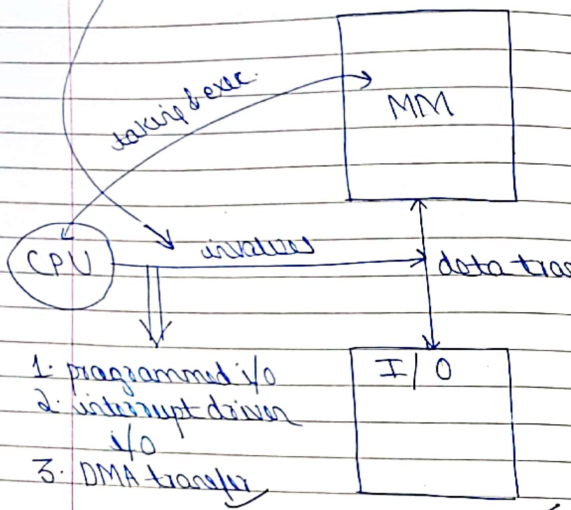
$B_C = B_L \times N \times \phi$

$C = L \times N \times \phi$ (RL)

Scanned with CamScanner

* I/O INTERFACING

- Data Transfer Modes.



1. programmed i/o
2. interrupt driven i/o
3. DMA transfer

→ Involvement of CPU is less. (should be)

Programmed I/O / Programmed Control

→ In this mode, i/o devices will not have direct access to memory.
→ Any transfer must involve the processor, that has to execute instructions repeatedly for:

1. Device Status — to check if ready.
2. I/O initialization:
   [Device address, Memory address, No. of words/ Buffer]
3. Data transfer
4. Status of transfer

→ Relatively more no. of instructions overhead.
→ due to transfer less words.
→ Inefficient mode of transfer.

Interrupt driven i/o (or) Interrupt initiated i/o.

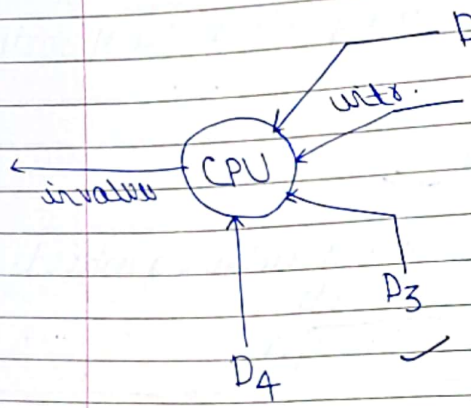
→ If ready sends interrupt. [if device is ready]
→ In this mode, when the device is ready; it sends an interrupt for data transfer.
→ Processor has to execute instructions for:

1. I/O initialization [DA, MA, N(w)]
2. Data Transfer
3. Status of Transfer

→ Relatively less instruction overhead compared to programmed i/o.
   → 1 instruction less overhead.

Scanned with CamScanner

→ Inefficient for large volumes of data transfer.



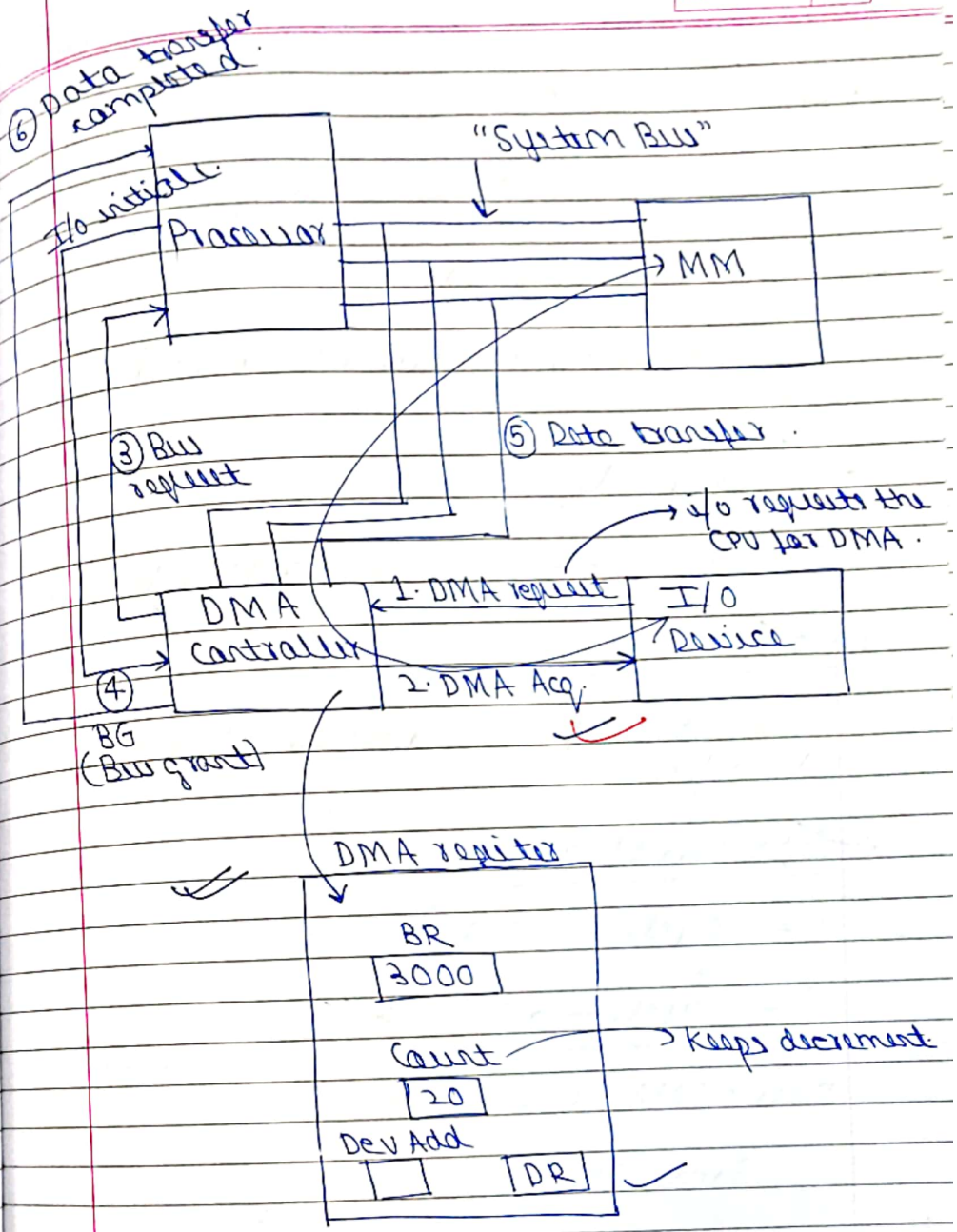
→ If a/ more devices simultaneously requests, processor implements priority driven i/o.

(Not suitable for large volumes of data transfer)

* DMA TRANSFER [Direct Memory Access]

→ for large volumes of data transfer b/w disks and memory, processor is relieved completely, using DMA controller.

* only the i/o initialization is done by CPU.



- i/o init. done by CPU, sent to DMA.

Scanned with CamScanner

* DMA Modes

- The way DMA takes control of bus:

1. cycle stealing mode:

- DMA returns the bus after a word transfer.

2. Block mode

DMA returns the bus after a block transfer.

3. Burst mode

DMA returns the bus after complete data transfer.

4. Interleaved mode

1/2 cycle → DMA
+
1/2 cycle → CPU

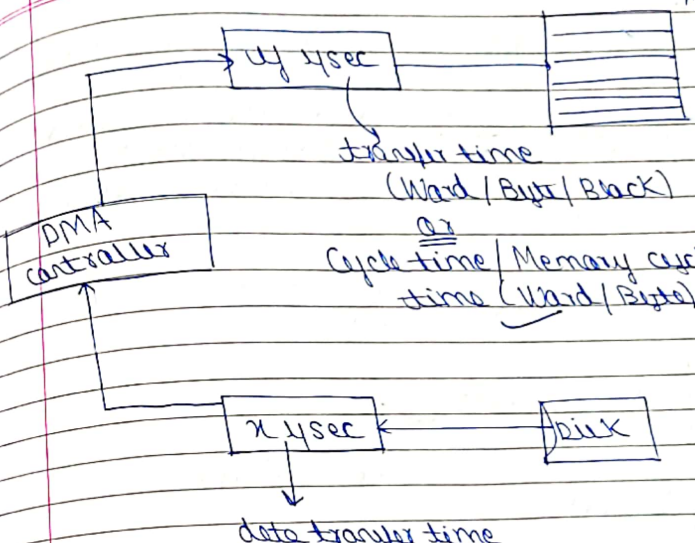
∴ During DMA, CPU → idle

∴ Some CPU performance low.

Time : 1K → 1000

Page No.: 129
Date:

MM



$$\%\,CPU = \frac{x}{x+y} * 100$$

$$\%\,CPU_{idle} = \frac{y}{x+y} \times 100$$

Scanned with CamScanner

Scanned with CamScanner

Q. Consider a device of 10 KBps is used in cycle stealing mode of DMA. When a 4 byte word is available, transferred into memory in 50 μsec. What is the performance of processor is reduced? (i/o)

Soln.

$$\%CPU_{idle} = \frac{y}{x+y} \times 100$$

$y = 50\ \mu sec$

⇒ 10KB ——— 1sec (10^6 μsec)
1 word (4B) ——— x

$$x = \frac{4 \times 10^6}{10 \times 10^3} = 400\ \mu sec$$

$$\%CPU_{idle} = \frac{50}{450} = \frac{100}{9} = 11.11\%$$

Q. A disc System has the following specifications (can be treated as i/o).

16 surfaces
128 tracks/surface
512 sectors/track
1KB/sector
3000 rpm
and a R/W Head. (movable R/W)

when a 4 byte word is available, transferred between memory and I/O; using DMA [Cycle Stealing mode]

If memory cycle time is 40 ns (y). What perc. of the processor is blocked?

Soln.:

$y = 40\ ns$.

$x = ?$

3000 rotations ——— 60×10^3 msec
1 rotn ——— x

$$x = \frac{60 \times 10^3}{3 \times 10^3} = 20\ msec$$

⇒ 1 rotn ⟶ 1 track (

1 track (512 * 1K) ——— 20 msec

1 word (4 BYTES) ——— x

$$x = \frac{20 \times 4}{512 \times 10^3}\ msec = \frac{4 \times 20}{512 \times 10^3} \times 10^6\ nsec$$

$x \Rightarrow 156.25\ nsec$

$$\%CPU_{idle} = \frac{y}{x+y} = \frac{40}{40+156.25}$$

$$\Rightarrow 0.203$$

$$\approx 20\%$$

Q The Storage area of a disc has inner diameter of 10cm & outer of 20cm. The maximum storage density is 1400 bits/cm and the disk rotates at 4200 rpm. The MM of the System is having 64 bit word and 1 μsec cycle time. (cycle stealing mode)

(i) What is the percentage of memory cycle stolen for transferring 1 word?

Soln.

$$P = \pi d = \frac{22}{7} \times 10 = 31.42 \text{ cm}$$

$\Rightarrow$ 43988 ← Track Capacity.

$\Rightarrow$ 44K bits

4200 rotations ——— 60×10^6 μsec

1 rotn ——— x

$$x = \frac{60 \times 10^6}{4200} \Rightarrow 14.28 \times 10^3 \text{ μsec}$$

$$\Rightarrow 14.28 \text{ msec}$$

44K bits ——— 14.28 msec

64 bit ——— x

$$x = \frac{64 \times 14.28}{44 \times 1000} \qquad y = 1 \mu s.$$

$\Rightarrow$ 0.020 msec

$\Rightarrow$ 20.7 μsec

$$\%CPU_{idle} = \frac{y}{x+y} = \frac{1}{20.7+1}$$

$\Rightarrow 0.046$

$\approx 4.6\% \times 5\%$

(ii) What is the processor performance?

$$\%CPU_{busy} = 95\%$$

$$\Rightarrow \frac{x}{x+y} \times 100$$

$$= \frac{20.77}{21.77} \Rightarrow 95.40\%$$

$\approx 95\%$

Scanned with CamScanner

$f \rightarrow$ Hz, $t \rightarrow$ nsec

Page No.: Date:

Q A Hard disk with a transfer rate of 10MBps is constantly transferring data to memory using DMA. The processor runs at 600 MHz and it takes 300 & 900 clock cycles to initiate and complete DMA transfer. If size of transfer is 20 KB, what is the percentage of processor performance reduced?

Soln:

$10\,MB \longrightarrow 10^6$ μsec

$20\,KB \longrightarrow x$

$$x = \frac{20 \times 10^3 \times 10^6}{10 \times 10^6}$$

$$\boxed{x \Rightarrow 2000\ \mu s}$$

Total clock cycles = 1200 clocks.

$f_c = 600$ MHz

clock cycle time $t_c = \frac{1}{600 \times 10^6} = \frac{10^6}{600} = \frac{1}{600}$ μsec

$$\Rightarrow y = \frac{1200}{600}\ \mu sec$$

$$\boxed{y = 2\ \mu sec}$$

Page No.: 135 Date: YOUVA

$$= \%\,CPU_{idle} = \frac{2}{2002} \times 100$$

$$\Rightarrow 0.09 \approx 0.1\,\%\ (Ans)$$

$$\%\,CPU_{busy} = 99.9\,\%$$

Q Consider a System in which DMA is used to transfer bytes between Memory and i/o using 16 bit count registers. How many times DMA request processor to transfer a file size of "15480 KB"?

[request when counter = 0].

K bit ⇒ 0 to $2^K - 1$. (2^K bits)

No. of requests = $\frac{15480 \times 2^{10}}{2^{16}}$

$1 \rightarrow 2^{16}$ B.

$x \rightarrow 15480$ KB

$$\Rightarrow \frac{15480}{64} = 242\ \text{Request}$$

Q Consider a Processor of 4 MIPS (Million Instructions per sec) (given) comp

(ii) what is peak data transfer Rate? using Programmed control, which requires 8 instruction overhead to transfer a word.

Scanned with CamScanner

D = no. of words / sec.

4×10^6 instructions ——— 1 sec
8 instruction ——— x

$x = \frac{8}{4 \times 10^6} = 2\,\mu sec$ → Toverhead.

2 μsec ——— per 1 word.
2 μsec ——— 1 word
10^6 μsec ——— x

$x = \frac{10^6}{2} \Rightarrow 0.5 \times 10^6$ words.

$\Rightarrow$ **D_p = 0.5 Million words / sec**

(iii) What is peak data transfer rate using interrupt mode, which requires 4 instructions overhead to transfer a word.

4×10^6 instr. ——— 1 sec
4 inst ——— x

$x = 1\,\mu sec$ → Toverhead

1 μsec ——— 1 word
10^6 μsec ——— x

D_{Pp} = 1 million words / sec

$\therefore$ **D_I = 1 million words / sec**

(iv) $P \propto D$ → Data Transf. Rate

$\frac{P_I}{P_p} = \frac{D_I}{D_P} =$

$P_I = \frac{1}{0.5} * P_p$

$\therefore$ **$P_I = 2 * P_p$**

→ Perform. of interrupt driven.

$P \propto \frac{1}{\text{Toverhead (Byte/Word)}}$

$\frac{P_I}{P_p} = \frac{T_p}{T_I}$

$\Rightarrow$ **$P_I = 2 * P_p$**

Scanned with CamScanner

Q. Consider a CPU in which each instruction execution completes in 7 clock cycles operated with 700 MHz. What is the data transfer rate using programmed control which requires 8 instructions overhead to transfer a byte (word).

Soln:

1 word ⟶ 8 instructn; time

1 instructn ⟶ 7 clock cycles

$f_c = 700\ MHz$

$t_c = \frac{1}{700}\ \mu sec$

↳ 1 clock cycle time

⇒ 1 instn ——— 7 clock cycles

$(7 \times \frac{1}{700})\ \mu sec$

1 instn ——— 0.01 μsec

8 instn ——— x

Toreh $x = 0.08\ \mu sec$ ⟶ 1 word.

⇒ 1 Byte 0.08 μsec ——— 1 byte

$10^6\ \mu sec$ (1s) ——— x

$x = \frac{10^6}{0.08}$

⇒ 12.5 Million Bytes / sec.

Q. Consider a device of 10 KBps is connected to a CPU, the data is transferred in byte wise. Let the interrupt overhead is 4 μsec. What is the performance gain of operating the device in interrupt mode over programmed control?

Soln:

$\frac{P_I}{P_P} = \frac{T_P}{T_I} = \frac{T_P}{4}$

10 KB ——— 1 Sec (10^6 μsec)

1 B ——— x

$x = \frac{10^6}{10 \times 10^3} = 100\ \mu sec$

$\frac{P_I}{P_P} = \frac{100}{4} = 25$

$\therefore \boxed{P_I = 25 * P_P}$

↳ 25 times better performance.

Scanned with CamScanner

Saturday

Lecture 4

* I/O Transfer:

- Synchronous transfer
  - comm clock (comp N/W)
  - Serial
    - N/W comm mostly used
    - Data link layer
  - Parallel (not used)
- Asynchronous transfer
  - comp N/W
  - Serial
    - i/o, CPU, MM
  - Parallel
    - CPM, MM, Printer, ALU

Sync. Char must be excluded

SYN	SYN	STX	DATA	ETX	CRC	SYN

Synchronous Characters

Asynch. Serial

Start bit	Parity bit	Char	Stop / end bit

8 bits

Sync bits [0, 1, 2] (included)

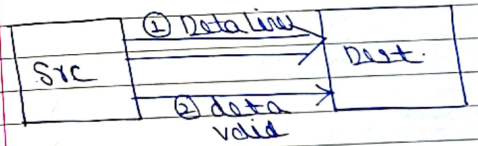
* Asynch. Parallel

2 protocols
1) Strobe Protocol
2) Handshake Protocol

* (i) Strobe Protocol

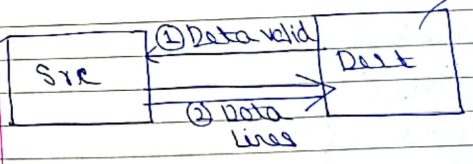
Src initiated strobe protocol

(ii)



Dest initiated strobe protocol

(iii)



Drawback → No acknowledgement from dest.

Page No. 41
Date

Lecture

* I/O Transfer:

- Synchronous transfer
  - Comm clock. (comp N/W)
  - Serial
    - N/W comm mostly used.
    - Data link layer
  - Parallel (not used)
- Asynchronous transfer
  - Serial
    - i/o; CPU; MM
  - Parallel
    - CPM, MM, Peripheral, ALU.

Syn: Char must be excluded

SYN	SYN	STX	DATA	ETX	CRC	SYN

Synchronous Characters

Asynch: Serial

Start bit	Parity bit	Char	Stop / end bit

8 bits

Sync. bits [0, 1, 2] (included)

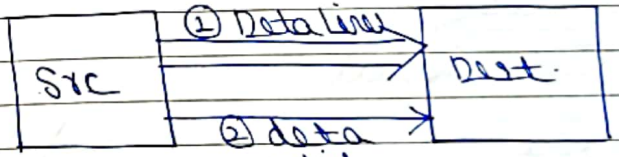
* Asynch: Parallel

2 protocols
1) Strobe Protocol
2) Handshake Protocol

* (1) Strobe Protocol

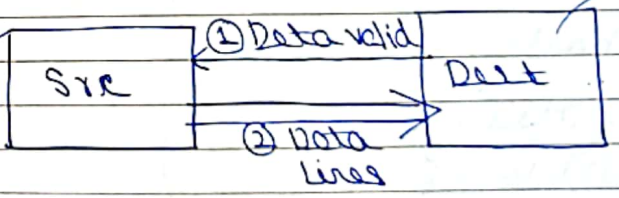
Src initiated strobe protocol

(i)



Dest initiated strobe protocol.

(ii)



Drawback → No acknowledgement from dest.

Scanned with CamScanner

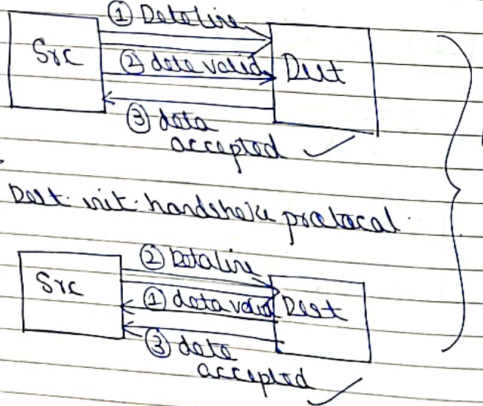
Synchronous	Asynchronous
• The SYN characters are excluded from data transfer rate	The SYN bits are included in Data transfer rate

- The rate at which serial information is transmitted is called batt rate/bit rate.

f = No. of bits/sec

(2) HandShake Protocal → req. & acknowledg.

(i) Src init. Handshake Protocal:



(ii) Dest. init. handshake protocal.

(After rec. data accepted/acknowledg.; data valid is disabled & data lines closed).

Q A Serial Transmission is employing 8 data bits, 1 parity bit & 2 Stop bits for each character. The minimum batt rate req. to sustain a transfer rate of 300 char/S

Asynch.

Soln

1 char → 11 bits
300 → x

Batt rate ⇒ (8 + 1 + 2) * 300 bits/sec

Each char size

⇒ 3300 bits/sec

Q Assume each character code consist of 8 bits. The no. of characters transmitted per sec with a batt rate of 2400 & 2 Stop bits.

Soln.

1 char → 10 bits
x → 2400 bits

$$T = \frac{2400}{(8+2)}$$

T ⇒ 240 char/sec

transfer Rate.

Scanned with CamScanner

Q A Serial transmission T_1 uses 8 information bits, 2 start bits, 1 stop bit and a parity bit for each character. A synchronous transmission T_2 uses 3 8bit Sync characters followed by 30 8bit information characters. The bit rate is 1200 in both cases. What are the transfer rates of T_1 & T_2

Soln:

$$T = \frac{1200}{8+2+1+1}$$ (char size)

$T_1 \Rightarrow$ 100 char/s — Serial Asynch.

1 char : 12 bits
x — 1200

Synchronous:

1 char : 8 bit
x

30 * 8 bits — 3 * 8 Syn bits
1200 bits — x

No. of Syn bits $= \frac{1200 \times 24}{240} = 120$

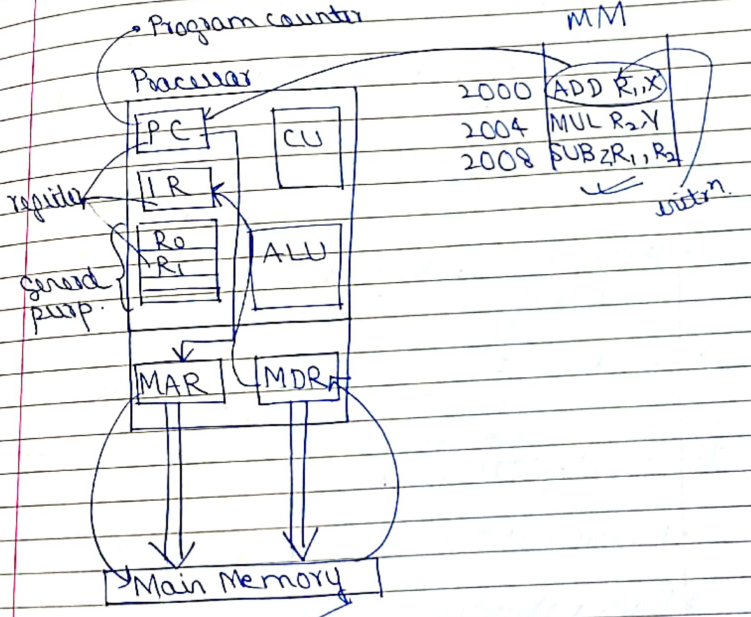
$$\Rightarrow T_2 = \frac{(1200 - 120)}{8}$$ (char size)

$= 135$ char/s

* MACHINE INSTRUCTIONS

1. BASIC OPERATIONAL CONCEPT

- how CPU exec 1 instructn



Scanned with CamScanner

* PC : (Program counter)

→ Keeps track / holds address of next instruction

* IR : (Instruction Register).

→ Holds current instruction code.

* MAR / AR : (Memory Address Register)

→ Holds address of memory to be referenced (read / write)

* MDR / DR / MBR : (Memory Data / Buffer Register)

→ Holds data to be read / write with memory.

* $R_0, R_1 \cdots R_{n-1}$ ⇒ (General Purpose registers)

* INSTRUCTION CYCLE

→ Every instn in CPU undergo some stages.

* Each instruction execution involves a set of phases / stages.

1) Instruction fetch (IF)

MAR ← PC [MAR ← 2000]
PC ← PC+1 [NEXT: 2004]
READ.
IR ← MDR

[Micro operation]

→ I

→ Getting current instruction in IR : Instn fetch.

2) Instruction decode (ID).

ADD R1, X → operand.

IR ⇒ | Opcode | Address Part |

| 1011 |

→ Decoder → Adder circuit activated.

"No Micro operation activated".

3) Operand Fetch (OF)

Read-X

MAR ← LOC-X
READ.
Ri/ALU ← MDR

} Micro-operation.

Scanned with CamScanner

destination

4) Execute and Store (EX)

→ Perform required ALU operation.

NOTE: The micro operations for execution depends on opcode (operation performed)

Store → write back (WB)

write (z)

↓

MAR ← LOC(Z)
MDR ← $R_1 - R_2$ / ALU / Ri } Micro operations
WRITE

5) Indirect cycle (IN)

MM

ADD R_1, 300

300 | 20

direct (or) absolute.

address of operand

(effective address).

ADD R_1, #300

indirect operands

300 | 850

850 | 20

EA

* Pointer to memory
* 2 mem cycles required.

(If indirect operands / pointers in prog.)

→ Indirect cycle

6) Interrupt cycle / Phase (IP)

- if interrupt occurs, CPU has to serve the interrupts.

Total instn cycle time.

1. Inst. Fetch (IF) } T_1
2. Inst. Decode (ID) }
3. Operand Fetch (OF) T_2
4. Execute & Store (EX) T_3
5. Indirect cycle (IN) T_4
6. Interrupt cycle (IP) T_5

* Each instruction execution involves a set of micro-operation to be implemented by the processor.

Micro-operations (Proc. perf only oper. in registers only)

VI

1. Arithmetic microoperations

(ADD R_1, 300) X [Not oper^n]

↳ instruction

Scanned with CamScanner

→ Micro-operation

→ Operation that are performed on registers only.

2. Logic micro-operation

$R_2 \leftarrow R_2 \wedge R_3$

3. Shift Micro opern

$R_2 \leftarrow R_3 << 2$

4. Register Transfer Micro operation:

$R_3 \leftarrow R_4$.

[takes 1 clock cycle]

Most micro-operations.

NOTE: [The micro operations of instruction cycle are mostly register transfers. each complete in 1 clock cycle].

{ eg: 1 instruction takes 7 clock cycles
mean,
7 micro-operations }

VI

8 micro operations.

Q. Consider a CPU in which execution of each instruction completes in 8 clock cycles, operated with 800MHz clock. What is the time taken to execute a program consisting of 16 instructions.

Soln:

⇒ Total CC = 16 × 8 = 128 clock cycle

$\frac{1}{800} \times 128 =$

Soln:

$f_c = 800\,MHz$

$T_c = \frac{1}{800}\,\mu sec \Rightarrow \frac{1}{800} \times 10^3\,nsec$

⇒ 1.25 nsec

1 clock cycle time

1 instruction ——→ 8 × 1.25 ns
16 inst' ——→ x

⇒ x = 16 × 8 × 1.25 = 160 nSec

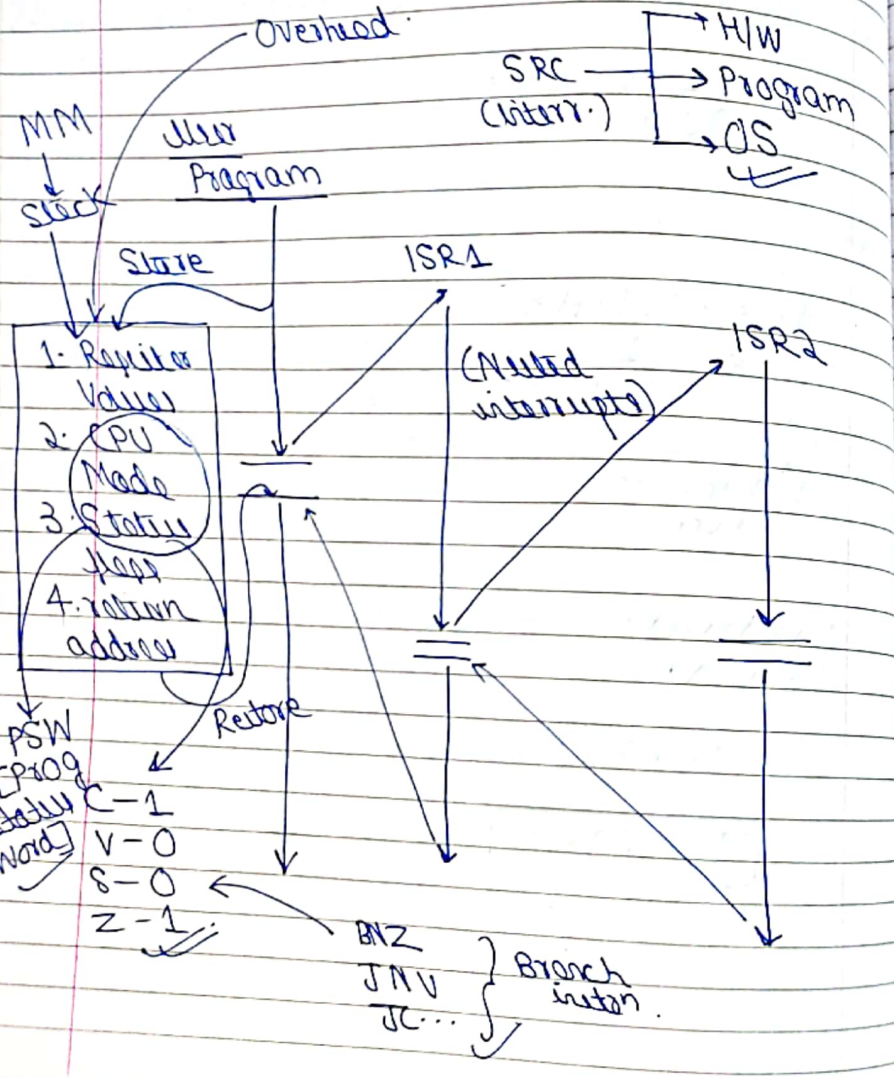
T = 160 ns

Scanned with CamScanner

ISR: Interrupt service routine

* INTERRUPT PROCESSING:

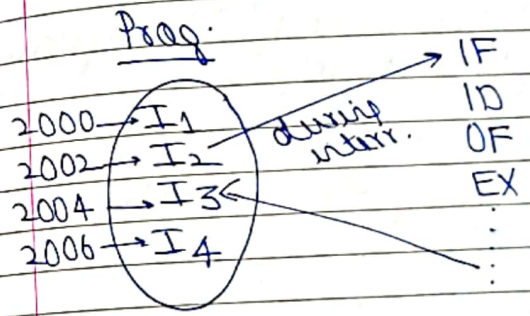
- current prog is suspended.
- Status flag updated (can test ALU op. (Branch with depend on Status flags)



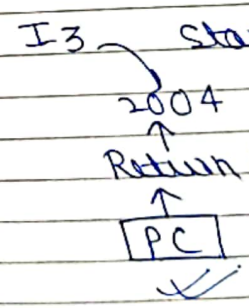
- From where, CPU has to resume executing the program after ISR: return address.

→ IP is an overhead to processor but it is its responsibility.

→ But Processor completes the curr. instruction (Prog suspended) V.I



Even though, CPU gets interr while performing I_2, it completes I_2

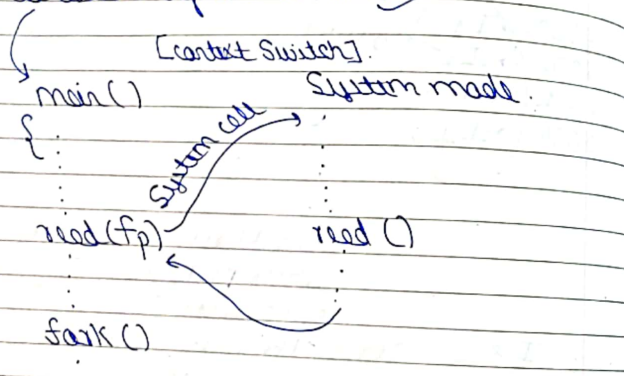


Scanned with CamScanner

Lecture 5

* A CPU has 2 Modes:

1. User/Non-privileged Mode
   → executes user program.
2. System/privileged/supervisory/Kernel Mode.
   → execute OS related services/OS programs to obtain System Services.



* The interrupts can be:

(ii) External/Hardware Interrupts: (H/W)
- Arised due to timing and i/o devices.
eg: i/o device request for data transfer. (printer, timer).

(iii) Internal interrupts: (Prog.)
- division by 0 error.
- 2 4bit no.; r > 4bits (overflow) } exceptions.
- exponent size > 8 bits.
→ Arised due to incorrect use of instructions and data.
eg: division by zero
Invalid opcode
register overflow
exponent overflow/underflow and so on.

(iv) Software Interrupt (OS)
→ Arised due to switching from user to System mode/vice-versa.
↳ context Switching.

Scanned with CamScanner

Q A CPU has 2 Modes (privileged and non-privileged). to change from one to another ____ interrupt is needed.

(Software)

Q A processor needs software interrupt ____.

(a) to invoke a Subprogram
(b) to test the interrupt System
(c) to obtain the System Services
(d) to implement co-routine.

Q A processor handles an interrupt ____

(a) As soon as it arrived.
(b) Periodic time intervals.
(c) After completion of a program.
(d) After completion of instruction cycle.

Q what is the possible operation performed on the following sequence of micro-operations:

MBR ← PC
MAR ← X
PC ← Y
Memory ← MBR

(a) Instruction Fetch.
(b) Operand Fetch
(c) Conditional Branch
(d) Initiation of Interrupt Service.

· OF doesn't involve PC.

During context Switch.

Memory ← PC
PC → Next instn Address
Next instn Address → return address
Stack ← return address

Q The following are the same of the events occur when an interrupt comes:

p: pushes the process status into the Stack.
q: finishes the execution of current instruction.
r: executes the ISR.
s: pops the status from the Stack.
t: loads the new PC value based on the interrupt.

return address.

q, p, t, r, s

Scanned with CamScanner

Find the sequence?

Q → P → T → R → S

"QPTRS"

PROBLEMS BASED ON INSTRUCTION SET

MODE VALUE

(Example)	(operand Type)	(Notation)	(Con
1. ADD R_1, R_2 ($R_1 \leftarrow R_1 + R_2$)	Register → 000	R_i	Value of operand
2. ADD $R_1, (R_2)$ ($R_1 \leftarrow R_1 + M[(R_2)]$)	Register Indirect → 001	(R_i)	Address of operand. eg: R_1 [20]; R_2 [350] → MM: 350 → 35 (EA)
3. ADD $R_1, 10(R_2)$ ($R_1 \leftarrow R_1 + M[(R_2)+10]$)	Scaled Register Indirect → 010	$X(R_i)$	Address of operand
4. Add $R_1, 10$ ($R_1 \leftarrow R_1 + 10$)	Immediate [used to initialise register] → 011	Const?	Value of operand.
5. ADD $R_1, 300$ ($R_1 \leftarrow R_1 + M[300]$) [to specify branch targets]	direct/ absolute → 100	const, any value	Address of operand [ADD R_1, X] [used to declare global variables]

eg: BNZ 350

Scanned with CamScanner

ALU (opr. ...

: Displac. > fast indirect.

6. ADD R_1, #300

Indirect — constant, any variable — address of operand

→ 2 mem. cycles required (Points to memory) — 101

7. ADD ~ Stack

CMA → AC [Comp.]

Implied (Zero Address Instruct.) — X — X (Some sym value opr.)

[Implicitly has operand] — 110

8. Displacement — Const. / any type operand — var — Address of operand

[always take 1 memory cycle + 1 ALU operation] — 111

∴ 8 combination ~ 3 bits ✓

Q Prep

↓

1. class ✓

(Last Pgs)

Q Consider the following program:

Instruct^n	Operation	Size (in words)	No of C: IF	EX
MOVI R_S, 1	$R_S \leftarrow 1$	2: 2×2		1
LOAD R_d, 1000(R_S)	$R_d \leftarrow M[(R_S)+1000]$	4: 4×2		4
ADDI R_d, 1000	$R_d \leftarrow R_d + 1000$	2: 2×2		1
STOR 0(R_d), 20	$M[(R_d)+0] \leftarrow 20$	4: 4×2		4
HALT	Stop	1: 1×2		1

(ii) Assume the contents of memory location 1000, 1001, 1020 are having the values 18, 1, and 16.

Identify the correct statement after the program is executed.

Scanned with CamScanner

(a) MC[1000] ← 20
(b) MC[1020] ← 20
(c) MC[1001] ← 20
(d) MC[1021] ← 20

Soln:

Rs = 1
R_d = MC[1000+1] = MC[1001] = 1
R_d = 1+1000 = 1001
MC[1001+0] ← 20.

(iii) For all instructions:

Fetch will take 2 clocks/word.
The execution of memory related consume 4 clocks & all others take 1 clock cycle each.

How many clock cycles required to compute the above program?

	IF	EX
⇒ I_1	4	1
I_2	8	4
I_3	4	1
I_4	8	4
I_5	2	1
	37CC.	

(ii) The CPU operates at 800 MHz
What is the time taken to complete the above prog.

⇒ $\frac{1}{800}$ μsec → 1 CC.

⇒ x → 37CC $t_c = 1/800$ μs.

⇒

$x = \frac{37}{800}$ μsec.

T_p = no. of clock cycles * t_c

⇒ ∴ 46.25 nsec

(iv) Let the size of a word be 64 bits.
How many bytes required to store the above program?

total no of words. $\frac{13 * 64}{8} = 104$ Bytes

(v) The above program is stored in byte addressable memory from location 1000 in decimal.
What return address is saved onto Stack if there is an interrupt during ADDI?

Scanned with CamScanner

Return Addr.

Byte addrs (Each Row → 1 byte)

Word addrs. (Each row → 1 word)

		$I_1 \to 1000$	
$I_1 \to 1000$		1001	
:		$I_2 \to 1002$	
:			
1015			
$I_2 \to 1016$			

2 words : $2 \times \frac{64}{8} = 16$ bytes.

4 W = $\frac{4 \times 64}{8} = 32$.

$I_3 \to 1048$.

2W = 16 bytes

$I_4 \to 1064$

I_1 : 1000 – 1015
I_2 : 1016 – 1047
I_3 : 1048 – 1063
I_4 : 1064 – 1095
I_5 : 1096 – 1103

} interrupt.

∴ Return Add = 1064

(vi) What return address is pushed onto stack if there is an interrupt during HALT?

→ Conceptually, no interrupt can occur during HALT.

(During HALT PC is not update).

PC ← PC + 1 X

∴ Return Address = 1096

- RA will be 1096 till OS updates it with next program.

If word Add:

during ADD : 1008 (RA)
during HAULT : 1012 (RA)

(vii) Consider the following program.

Scanned with CamScanner

Instruction	Operation	Size (words)
MOV R1, 5000	$R_1 \leftarrow M[5000]$	2
MOV R2, (R1)	$R_2 \leftarrow M[(R_1)]$	1
ADD R_2, R_3	$R_2 \leftarrow R_2 + R_3$	1
MOV 6000, R_2	$M[6000] \leftarrow R_2$	2
HAULT	Stop	1

(i) The clock cycles needed for various operations is:

Register to/from memory : 3CC
Add with both operands in Registers : 1CC
Instruction Fetch & Decode : 2 clocks/word

How many clock cycles required?

(ii) The above program is stored in byte addressable memory. What return address is pushed onto stack if there is an interrupt during hault?

(Word Size = 32 bits) → 4 Byte
(Start Address = 1000)

(ii) Byte Addressable Memory

SA = 1000.

I_1 = 1000 – 1063 (32 X 2 = 64)
I_2 = 1064 – 1095
I_3 = 1096 – 1127
I_4 = 1128 – 1191
I_5 = 1192 – 1223

∴ During HAULT : RA = 1192. Ans

(ii)

		IF ID	R-M	ADD
I_1	2	4	3	-
I_2	4	8	3	-
I_3	2	4	-	1
I_4	4	8	3	-
I_5	1	2	-	-

∴ CC = 7 + 11 + 5 + 11 + 2 ← wrong value.

CC ⇒ 36 X

	F & D	RM	ADD
I_1	4 4	3	-
I_2	8 2	3	-
I_3	4 2	-	1
I_4	8 4	3	-
I_5	2	-	-

7 + 5 + 3 + 7 + 2
⇒ 24

Scanned with CamScanner

I_1 : 1000 - 1007 ✓
I_2 : 1008 - 1011
I_3 : 1012 - 15
I_4 : 1016 - 23
I_5 : 1024 - 27

I024 = RA ✓

Lecture 5

9. Consider the following program Segment:

Instruction	Operation	C(word)
MOV R1,3000	R1 ← M[3000] ✓ ①	2
LOOP:	-	-
MOV R2,(R3)	R2 ← M[(R3)] ✓ ②	1
ADD R2,R1	R2 ← R1 + R2	1
MOV (R3),R2	M[(R3)] ← R2 ✓ ③	1
INC R3	R3 ← R3 + 1	1
DEC R1	R1 ← R1 - 1	1
BNZ LOOP	Branch if not zero Loop;	2
HALT.	Stop	1

(ii) Assume the contents of memory location 3000 is 10. M[3000] ← 10 in word addressable memory. The no. of memory references for accessing data in executing the program completely.

Soln

R1 ← 10

1 + (2 × 10) = 21 memory references.

① happens one time ; (② & ③ occurs till loop ends)

Scanned with CamScanner

word Addressable

(iii) Let the size of a word is 32 bits. What return address is saved onto stack if there is an interrupt during INC R3. (SA = 1000)

Soln.

I1 : 1000 - 1001
I2 : -
I3 : 2 - 3
I4 : 3 - 4.
I5 : 1005
I6 : 1005
I7 : 1006 → Return Address.
I8 : 1007 - 08
I9 : 1009 ✓

[Assume]

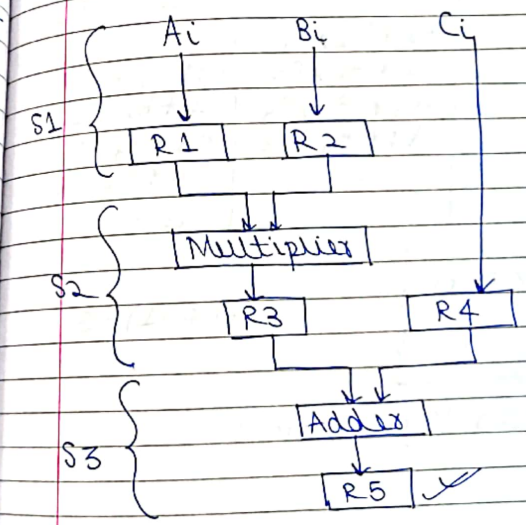
PIPELINING

* 1. ARITHMETIC PIPELINE

- Eq:

$D_i \leftarrow A_i * B_i + C_i$ → 4 instructions

where i = 1 to 4



∴ 1 stage : 1CC ∴ 3 stages : 3CC

∴ Total = 12CC (without pipeline)

Scanned with CamScanner

Assume each Stage completes in 1 clock cycle.

let: n → no. of tasks/instructions (n=4)
K → no. of stage/segments (K=3)

Using sequential/non-pipelined execution:

No. of CC = n * K ∴ 4*3=12 CC.

* Using Pipelined execution: → Parallel execution:

Clock	S1	S2	S3
1	A1, B1	–	–
2	A2, B2	$A_1 * B_1 + C_1$	–
3	A3, B3	$A_2 * B_2 + C_2$	$A_1 * B_1 + C_1$
4	A4, B4	$A_3 * B_3 + C_3$	$A_2 * B_2 + C_2$
5	–	$A_4 * B_4 + C_4$	$A_3 * B_3 + C_3$
6	–	–	$A_4 * B_4 + C_4$

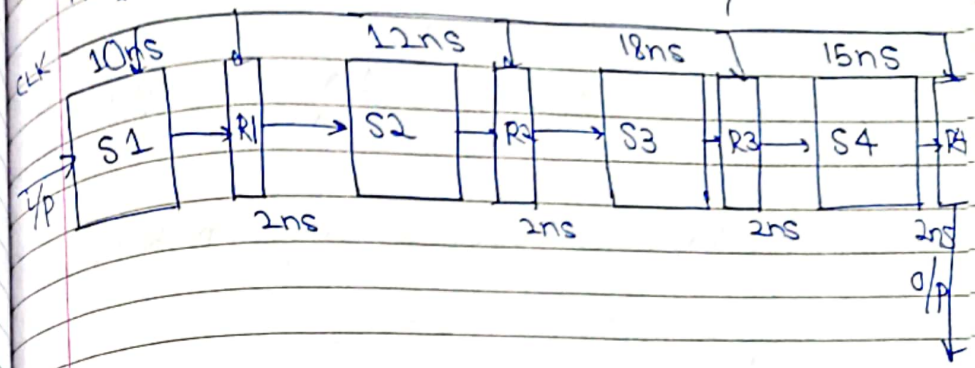
No. of Clock Cycles:

K + (n-1) → 3+(4-1)=6.

∵ 1st stage takes K clock cycles.

VH

A 4 Segment Pipeline can be: → Synchr.



→ A task is the total operation performed by passing through all the stages of the pipeline.

→ Stage delay (ti):

Processing time in a Stage;
It varies from 1 stage to another.

→ Inter Stage delay (td):

Inter Stage transfer of data, it is saved between the Stages.

VI

→ The time period for clock cycle in Pipelined execution

· tp = max(ti) + td

· max{ti} >>>> td.

∴ tp = max{ti}

tp = 1 CC time.
↳ 1 stage comp. time

Scanned with CamScanner

→ Frequency of the Pipeline :

$$f_p = \frac{1}{t_p}$$

t_p : 1 clock time/ 1 task completion time.

→ Time taken in Pipeline with (n, k) :

$$T_p = (k + (n-1))t_p$$

T_p = no. of clock cycles $* t_p$.

→ Dividing a task into stages, all tasks must operate simultaneously in Pipelining.

→ Providing parallel processing in a uni-processor System; thus improving throughput.

$$\text{Throughput} = \frac{\text{No. of tasks/ jobs}}{\text{sec}}$$

(a) Asynchronous Pipeline :

→ Data flow along the pipeline Stage is controlled using Handshake Protocol.

(i) Synchronous Pipeline (by default) :

- On a common clock, all the registers transfer data to the next Stage simultaneously.
- Also t_{clk}, $S1 \rightarrow S2$

→ Speed up (S)

$$S = \frac{\text{Time without pipeline}}{\text{Time with pipeline}}$$

Ex- Let $t_n \Rightarrow$ time to complete a task/ instruction in non-pipeline.

$$S = \frac{n * t_n}{(k+(n-1)) * t_p} \quad \text{---(1)}$$

→ steady computation time

time for 1 task

$t_n \approx K t_p$ ✓

time for 1 task.

Substitute in (1).

$\Rightarrow S = \frac{n * K t_p}{(K+(n-1)) * t_p}$

$S = \frac{nK}{K+(n-1)}$ ——— (2) ✓

* For large no. of tasks / instructions:

$K+(n-1) \longrightarrow n$.

Substitute in (1)

$\Rightarrow S = \frac{n t_n}{n t_p}$

$S = \frac{t_n}{t_p}$ ——— (3) ✓

Substitute in (2):

$\Rightarrow S = \frac{nK}{n} = K$.

$S = K$ ——— (4).

∴ K X faster than non-pipelined.

NOTE: Max Speedup using a Pipeline is equal to no. of stages.

$S_{max} = S_{ideal} = K$ ✓

* Efficiency of the Pipeline:

$E_K = \frac{S}{K}$ ✓

$E_K = \frac{nK / K+(n-1)}{K}$

$E_K \Rightarrow \frac{n}{K+(n-1)}$ ✓

i, $S = K \therefore E_K = 100\%$ $\because S = S_{max}$ ✓

* Throughput of the Pipeline. → no. of tasks / pipelined time.

$H_K = \frac{n}{(K+(n-1))t_p}$ ✓

$H_K = E_K * \frac{1}{t_p}$

$H_K = E_K * f_p$

Scanned with CamScanner

Q The number of clock cycles needed to perform 100 tasks in a 6 segment pipeline

No of CC = K + (n-1)
⇒ 6 + (100 - 1)
⇒ 105

No of CC = 105

Q For a given 6 Segment Pipeline, what is the Speed up for 100 tasks.

$$S = \frac{nK}{K+(n-1)}$$
$$\Rightarrow \frac{100 \times 6}{105}$$
$$S = 5.71$$

Q For a given 6 Segment Pipeline, what is the efficiency for 100 tasks.

$$E_K = \frac{n}{K+(n-1)}$$
$$\Rightarrow \frac{6}{105}\,100 = 0.95$$

∴ 95% efficient

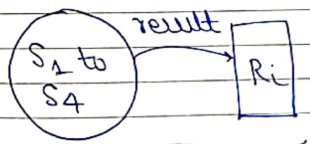
Q During floating point arithmetic, the time delay in 4 stage of a pipeline is 60ns, 70ns, 90ns & 80ns. It has inter stage delay of 10ns.

(i) what is the time period for clock cycle?

$$t_p = \max\{t_i\} + t_d$$
$$\Rightarrow 90 + 10 = 100$$
$$\therefore t_p = 100\text{ns}$$

(ii) what is the Speed up?

$$S = \frac{t_n}{t_p} \qquad \text{(don't know)}$$
$$\Rightarrow \frac{60+70+90+80+10}{90+10} \quad \text{(10 = overhead)}$$
$$\Rightarrow \frac{310}{100} = 3.1$$



non-pipeline

Scanned with CamScanner

(iii) What is the max Speed up?

$S_{max} = K$

$\therefore K = 4$ $\boxed{S_{max} = 4}$

Q Consider a Synchronous pipeline with 4 stages, where each stage takes a uniform delay of 20ns and has buffer overhead of 5ns.

(i) what is the clock frequency of the pipeline?

Soln: $f_p = \frac{1}{t_p}$

$t_p = \max\{t_i\} + t_d$

$\Rightarrow 20 + 5 = 25$ ns.

$f_p = \frac{1}{25 \times 10^{-9}} = \frac{10^9}{25}$ Hz

$\Rightarrow$ 40 MHz.

$\boxed{f_p = 40 \text{ MHz}}$

(iii) what is the speed up?

$S = \frac{t_n}{t_p} = \frac{85}{25}$

$\frac{t_n}{t_p} = \frac{(20+20+20+20)+5}{25} \Rightarrow 3.4$

Q Consider a Synchronous pipeline with 5 stages. The delay of S3 is twice to that of S2 and equal to that of S5. S3 & S4 are having same delay as that of S1. The delay of S1 = 10 nS. What is the speed up?

S1 = S3 = S4 = 10nS.

S2 = 5nS

S5 = 20nS

$S = \frac{t_n}{t_p} = \frac{55}{20} \Rightarrow 2.75$ · Ans

non-pipeline

Q Consider a CPU in which each instruction execution completes in 4 clock cycles, operated with 2.5 GHz. The same processor is transformed into pipelined processor with 5 stages, operates at 2.5 GHz. What is the Speed up?

Scanned with CamScanner

tn: time taken by 1 CC (pipelined)
tp: time taken by

$t_p = \frac{1}{f}$

$S = t_n / t_p$

$\Rightarrow S = f_c = 2.5\,GHz, \; t_c = \frac{1}{2.5 \times 10^9}$

$\Rightarrow \frac{1}{2.5} = 0.4\,ns$

∴ 1 instn → 4 CC

∴ (1.6 ns) → t_n

$\Rightarrow t_p = \frac{1}{f_t} = \frac{1}{2}\,ns = 0.5\,ns$ → t_p

$S = \frac{t_n}{t_p} = \frac{1.6}{0.5} = 3.2$

Q. Consider a pipelined processor with 4 stages operated at 800 MHz. What is the time taken to perform 20 tasks. → n

Soln.

$t_p = \frac{1}{f_p}$

$\Rightarrow \frac{1}{800 \times 10^6}$ → 1 CC

$\Rightarrow 1.25\,ns$

$T_p = (K + (n-1)) * t_p$

$\Rightarrow (4 + 19) * 1.25 = 28.75\,ns$

Comparing 2 diff. pipelines.

Page No.: 183 Date:

Q. Consider 2 Pipelines: A & B. → K

→ Design A is having 5 stages with delays.

$t_i = 4\,ns, 5\,ns, 4\,ns, 3\,ns, 5\,ns.$

and

with buffer overhead of 1 ns.

→ Design B is having 9 Stages with a uniform delay of 2 ns.

(ii) How much time is saved using design B. over A for 100 instruction? → n

Soln.

$T_A = ((K + (n-1))\, t_p$

$\Rightarrow (5 + (100-1))\, 6 = 624\,ns$

$T_B = (9 + (100-1)) * 2 = 216\,ns.$

∴ 408 ns saved.

(iii) What is the Speed up?

$S = \frac{T_A}{T_B} = \frac{624}{216} \Rightarrow 2.89$

$\boxed{S = 2.89}$

Scanned with CamScanner

same pipeline.

Q $K=5$; $n=100$

$t_i = 4ns, 5ns, 13ns, 6ns, 5ns$

$t_d = 1ns$

7ns 6ns

II.

what is the speed up?

VI.

Soln

$$S_1 = \frac{(5+99)*14}{(6+99)*8}$$

$$\Rightarrow \frac{1456}{840} \Rightarrow 1.73$$

$\boxed{S = 1.73}$

Q For a given task:

$T_1 \longrightarrow$ Time taken in pipeline.

$T_2 \longrightarrow$ Time taken in non-pipeline.

a) $T_1 = T_2$

b) $T_1 < T_2$

VI

c) $T_1 > T_2$

d) None

$n = 1$

$T_1 = (K + (n-1))\, tp$. $\quad T_2 = tn$.

$\Rightarrow K\, tp$.

$tn = 10+12+18+15+2$

$\Rightarrow 57\, ns$

$T_1 = 4*(18+2) = 20\, ns$

INSTRUCTION PIPELINE

- Overlapping various stages of instruction cycle for several instructions is instruction pipeline.

- The objective is $\boxed{CPI_{avg} = 1}$

(clocks per instruction)

- A 4 Stage instruction pipeline can be:
(4 instr.)

Clock Cycle →	1	2	3	4	5	6	7
I1	IF	ID	OF	EX			
I2		IF	ID	OF	EX		
I3			IF	ID	OF	EX	
I4				IF	ID	OF	EX

Each Stage take 1 CC.

completely overlapped pipeline.

4 instr.

4 Stage – 4 instr. [Each Stage → 1 CC]

$(4+3) = 7$

IF for I2 in 2nd CC.

Scanned with CamScanner

(i) Consider an instruction pipeline with 4 stages the clock cycles needed by the stages for each instruction is:

	F	D	E	S
I1	2	1	2	2
I2	1	3	3	2
I3	2	2	2	2
I4	1	3	1	1

→ Completely overlapped pipeline.

(ii) The no. of clock cycles needed in sequential non-pipelined execution.

Soln:

7+9+8+6 = 30 clock cycles

{I1 I2 I3 I4}

(iii) How many clock cycles needed in pipeline execution? ⇒ 14.

$n_t = K + (n-1)X$ [each stage → 1CC]

	1	2	3	4	5	6	7	8	9	10	11	12	13
I_1	F	F	D	E	E	S	S						
I_2	-	F	D	D	D	E	E	E	S	S			
I_3		-	F	F	-	D	D	-	E	E	S	S	
I_4			-	-	F	-	(-)	D	D	D	E	-	S

→ stall cycle

(iv) What is the Speed up?

Ex/Ep. $S = \frac{30}{14} \frac{(np)}{(p)} = 2.14$

(iii) What is the utilisation factor?

$u = \frac{14}{30}$ ← clocks used / ← total clocks.

$u \Rightarrow 0.46$

(iv) The pipeline is operated with 800 MHz. what is the time taken to complete 4 instructions?

Soln

1 inst 4 instr → 14 CC.

⇒ $\frac{1}{800 \times 10^6} = 1.25$ ns (tp)

(time for 1 CC.)

$T_p = 1.25 * 14 = 17.5$ ns

Scanned with CamScanner

* STALL CYCLE

→ During it, no meaningful operation performed for an instruction.
→ A stall cycle will arise due to:

(i) different clock cycles needed by the stages for each instruction.
(ii) maximum buffer overhead.
[next stage waits for.]
(iii) Instruction pipeline dependencies

$$S_{eff} = \frac{S_{ideal}}{1 + Stall\ freq * Stall\ cycles}$$

↳ effective speed up

$S_{ideal} = K$ (V-I)

$$\therefore S_{eff} = \frac{K}{(1 + stall\ freq * Stall\ cycles)}$$

Q Consider an instruction pipeline with 5 stages, it has 2 stall cycles penalty for every memory dependency. If there are 20% memory references. What is the speed up?

$$S_{eff} = \frac{5}{1 + 2 * 0.2}$$

$$\Rightarrow \frac{5}{1 + 0.4} = 5/1.4 = 3.57$$

[Pipeline with no stall cycles]
↳ Completely overlapped pipeline.

Q Consider a 4 stage instruction pipeline to execute a loop.

```
for(i=1; i<=100; i++)
{
    I1;
    I2;
    I3;
    I4;
}
```

The clock cycles needed by the stages for each instruction:

1 2 3 4 5 6 7 8 9 10 11 12 13 14

	S_1	S_2	S_3	S_4
I_1	1	2	1	2
I_2	2	1	2	1
I_3	1	1	2	1
I_4	2	1	2	1

Scanned with CamScanner

(ii) The output of I_1 for $i=2$ is available after ___ clock cycles

	1	2	3	4	5	6	7	8	9	10	11	12	13
I_1	S_1	S_2	S_2	S_3	S_4	S_4							
I_2		S_1	S_1	S_2	S_3	S_3	S_4						
I_3			S_1	S_2	–	S_3	S_3	S_4					
I_4				S_1	S_1	S_2	–	S_3	S_3	S_4			
I 1				–	–	S_1	S_2	S_2	–	S_3	S_4	S_4	
i=2													

∴ 13 clock cycles

5th instruction.

(iii) The no. of clock cycles needed to execute the loop:

```
for (i=1 to 2) do
{
    I1;
    I2;
    I3;
    I4;
}
```


1 2 3

13 14 15 16 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	S_1	S_2	S_2	S_3	S_4	S_4											
I_2			S_1	S_1	S_2	–	S_3	S_3	S_4								
I_3				S_1	S_2	–	–	S_3	S_3	S_4							
I_4					S_1	S_1	S_2	–	–	S_3	S_3						

continuation of prev.

S_4

18 clock cycle

Q Consider an instruction pipeline with 4 Stages.

IF, OF, & WB ⟶ 1 clock cycle each
EX ⟶ 45 instruction ⟶ 3 clocks
35 instr^n ⟶ 2 clocks
20 instr^n ⟶ 1 clock

How many clock cycles required?

Soln:

total 100 instr.

Scanned with CamScanner

⊙ K=4, n=100

No. of clock cycles = K+(n-1)

⇒ 4+(100-1) = 103

If each stall = 1 clock cycle

VI

Excess for 45 = 45×2 = 90 (1 covered by form)
Excess for 35 = 35×1 = 35
Excess for 20 = 20×0 = 0

∴ 228 clocks

* INSTRUCTION PIPELINE DEPENDENCIES (CONFLICTS/HAZARDS)

ready to stall cycle

The solution can be:

(ii) Hardware
- Pipeline h/w takes care

(iii) Software (S/W)
- The compiler in which we are writing, takes care.

⇒ 3 types of dependencies:

(1) Data dependency → I_3 depends on result of I_2

Eg:
I_1: $R_0 \leftarrow R_1 + R_2$
I_2: $R_2 \leftarrow R_3 + R_4$
I_3: $R_5 \leftarrow R_6 * R_2$
I_4: $R_7 \leftarrow R_8 - 1$

I_1, I_2 → Sequential
I_2, I_3 → Sequential
I_3, I_4 → parallel

→ If an instruction depends on the result of its previous instruction, it leads to data dependency.

	1	2	3	4	5	6	7
I_1	IF	ID	OF	EX			
I_2		IF	ID	OF	EX		
I_3			IF	ID	-	OF	EX
I_4							

Stall cycle (data dependency)

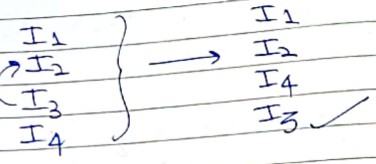
→ The solution can be: → needs parallel stat.

1. Dynamic scheduling (H/W & S/W)

→ Rearranges the instructions such that the conflict is resolved.
→ Instruction after dependency need parallel statements.

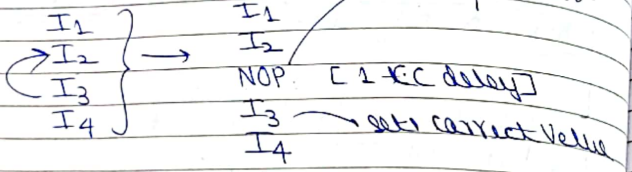
Scanned with CamScanner

NOP↑, CC↑.



2. Delayed load (S/W)

- compiler takes care.



→ Rearranges instructions by inserting appropriate number of NOPS by the compiler.

↳ no result

Drawback:

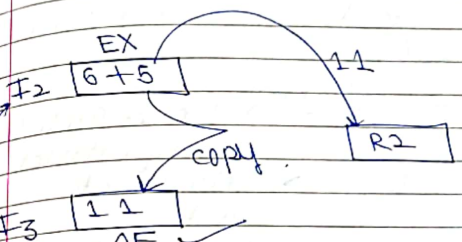
Increases no. of clock cycle. → CC↑ (throughput↓)

→ Implemented in RISC processor.

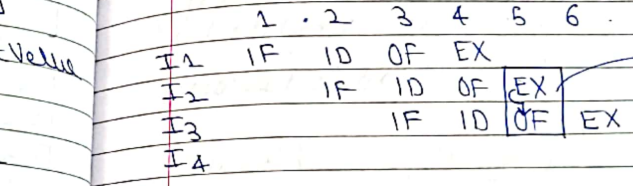
↳ [Reduced Instruction Set Computer]

VE

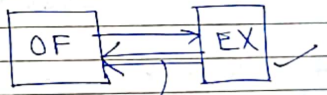
3. Operand Forwarding (H/W) [default soln]



(copy is sumult. sent to prev. operation of next instn).



→ Operand forw.



operand forw. (H/W)

→ saves extra clock pulse.

→ done in the h/w.

Q Consider an instruction pipeline with 4 stages.

[Fetch (F), Decode (D), Execute (E), and Write Back (W).]

The stages F, D & W ⟶ take 1 clock cycle for every instruction.

E ⟶ 3 clocks ⟶ *
and
1 clock ⟶ + & −.

If operand forwarding is used, How many clock cycles required to complete the following program?

↳ data dependency.

I1 ADD (R2), R1, R0
I2 MUL (R4), R3, (R2)
I3 SUB R6, R5, (R4)

3 instrn
2 dependencies.

	1	2	3	4	5	6	7	8	9	10
I1	F	D	E	W						
I2		F	D	E	E	E	W			
I3			F	D	–	–	E	W		

R2 after 4th CC.

↳ 8 clock cycles needed

F, D, E, W

↳ operand forwarding.

Q Multiply

MUL (R5), R0, R1
DIV (R6), R2, R3
ADD (R7), (R5), (R6)
SUB R8, (R7), R4

3 dependencies

Stages: Instruction Fetch: IF ⟶ 1 clock
& Decode
Operand Fetch: OF ⟶ 1 clock
Perform operation: PO ⟶ 1 clock cycle for +, −
(ex) 3 CC: *
5 CC: :−
Write Back (WB) ⟶ 1 clock

If operand forwarding is used. How many clock cycles required to complete the program?

↳ from PO → OF. (given)

Scanned with CamScanner

	1	2	3	4	5	6	7	8	9	10	11	12	13
I1	IF	OF	PO	PO	PO	WB							
I2	-	IF	OF	-	-	PO	PO	PO	PO	PO	WB		
I3			IF	-	-	-	-	-	-	OF	PO	WB	
I4				IF	-	-	-	-	-	-	OF	PO	WB

∴ 13 clock cycles required.

* Data dependency can be:

(1) RAW Hazard or True Dependency

(Read After Write)

ADD R7, R5, R6.
SUB R8, R4, R7.

$R_1 \cap P_2 \neq \phi$

(2) Anti-dependency (WAR Hazard)

ADD R5, R6, R7 — R: Write after Read.
SUB R7, R4, R3 — W

$D_1 \cap R_2 \neq \phi$

(3) Output dependency (WAW Hazard)

ADD R7, R6, R5
SUB R7, R4, R3

$R_1 \cap R_2 \neq \phi$

(4) No dependency (RAR Hazard)

ADD R7, R6, R5
SUB R8, R6, R4 → not a dependency

$D_1 \cap D_2 \neq \phi$

Q What are the possible Hazards for the following instructions:

Domain (D_1)
Range (R_1)
ADD (R5, R6, R7)
SUB (R5, R5, R4)

R_2 D_2

VI

RAW: $R_1 \cap D_2 = \{R_5\} \cap (R_5, R_4) = \{R_5\} \neq \phi$.
WAR: $D_1 \cap R_2 = R_6 R_4 \cap R_5 = \phi$.
WAW: $R_1 \cap R_2 = R_5 \cap R_5 = R_5 \neq \phi$.

∴ RAW & WAW Hazards will be possible.

Scanned with CamScanner

2) CONTROL DEPENDENCY

- Branch instruction leads to control dependency.
- The target of Branch is not allowed until the branch instruction is executed completely.

Eg:

```
I1
I2
I3 : BNZ I7 ;
I4
:
:
I7
```

⇒

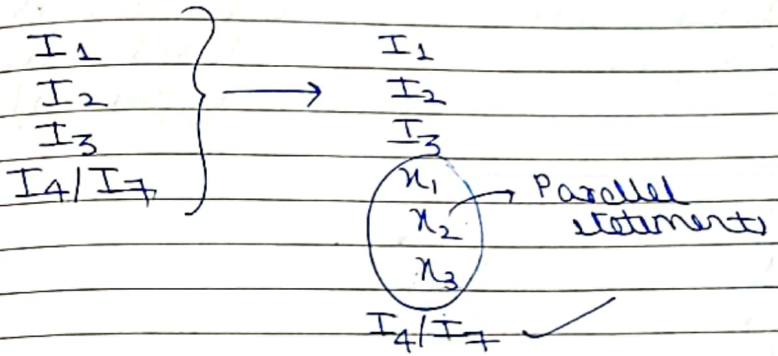
	1	2	3	4	5	6	7
I_1	IF	ID	OF	EX			
I_2		IF	ID	OF	EX		
I_3			IF	ID	OF	EX	
I_4/I_7			–	–	–	–	IF

Stall cycles (control dependencies)
[3 delays here]

* The Solution can be:

1) Dynamic Scheduling (H/W or S/W)

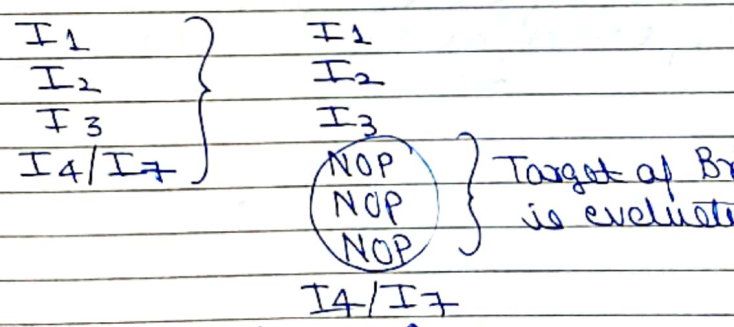
→ waiting for I_3 to complete.



– Waiting for Branch init. to complete its execution.

3) Delayed load/Branch (S/W) → RISC processor

– NOP inserted



= ↑ Branch, ↑ NOP, ↑ CC.

Scanned with CamScanner

Branch Prediction

- Static BP
  - [Prediction never changes]
    - Branch Always Taken
      - [Target = [7]]
    - Branch Never Taken
      - [Target = I4]
- Dynamic BP:
  - [Prediction changes based on outcome of previous branches]

→ If Prediction goes wrong, it leads to Branch penalty. (Stall cycles).
If correct, no Branch penalty.

Control dependency

→ implemented using 2 techniques

- with prediction
  - Prediction correct
    - Stall cycles = 0
  - Prediction wrong
    - Stall cycles = x
- without prediction
  - Stall cycles = x
    - parallel stat.
    - NOP statements

Stall cycles for Control dependency (VII)

- K=5
  - $(S_1 - S_5)$
  - (IF, ID, OF, EX, WB]

I_3: S_1 S_2 S_3 S_4 S_5

14/17: — — — — S_1

4 Stall cycles

I_3: IF ID OF EX WB

I_4/I_7: — — — IF

3 Stall cycles.

after exec of Branch Inst.

Scanned with CamScanner

∴ Branch ... leads to SC.

Page No.: Date: YOUVA

(i) Consider an instruction pipeline with 5 stages. It allows overlapping of all instructions except branch type. Let there are 20% Branch instructions and the pipeline is operated with 800 MHz.

(ii) What is the Speed up?

Soln:

$$S = \frac{K}{(1 + \text{Stall freq} * \text{Stall cycles})}$$ (K − 1.)

$$\Rightarrow \frac{5}{1 + 0.2 * 4}$$ (4 → ready: stall cycle = 4)

$$\Rightarrow 5/1.8 = 2.7$$ ✓

(iii) What is avg instruction time? (1 / throughput)

$$T_{avg} = \left(1 + \text{Stall freq} * \text{Stall cycles}\right) * t_p$$ ✓

$f_p = 800\,MHz$

$t_p = 1.25\,ns$

$$T_{avg} = (1 + 0.2 * 4) * 1.25$$

$$T_{avg} \Rightarrow 2.25\,ns$$ ✓

$\tau = \frac{S}{K} \times \frac{1}{T_p}$ $\quad \frac{IPK}{S}$

Page No.: 205 Date: YOUVA

(iii) What is the time taken for 10 million instructions?

1 inst ⟶ 2.25 ns

10×10^6 inst ⟶ x

$x = 10^7 * 2.25 * 10^{-9}$

$x \Rightarrow 22.5\,msec$ ✓

K = 5 (stages) (with no Prediction).

Defn of stage given → [IF, ID, OF, EX, WB]

$t_i \Rightarrow 6ns, 7ns, 9ns, 8ns, 7ns$.

$t_d \Rightarrow 1\,ns$

Program:

I1
I2.
I3 : BNZ I9 ; → Branch is Taken
I4
:
I9
I10

What is the time taken to complete the Program?

Scanned with CamScanner

∴ No Prediction :

T_p = no. of clock cycles * tp

tp = max(ti) + td = 10 ns.

	1	2	3	4	5	6	7	8	9	10	11	12	13
I_1	IF	ID	OF	EX	WB								
I_2		IF	ID	OF	EX	WB							
I_3			IF	ID	OF	EX	WB						
I_9				—	—	—	IF	ID	OF	EX	WB		
I_{10}								IF	ID	OF	EX	WB	

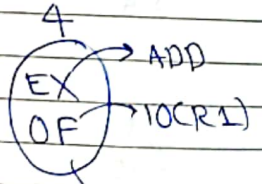
∴ 12 clock cycles

⇒ T_p = 120 ns

3) STRUCTURAL DEPENDENCY (Resource Conflict)

h/w provides Soln.

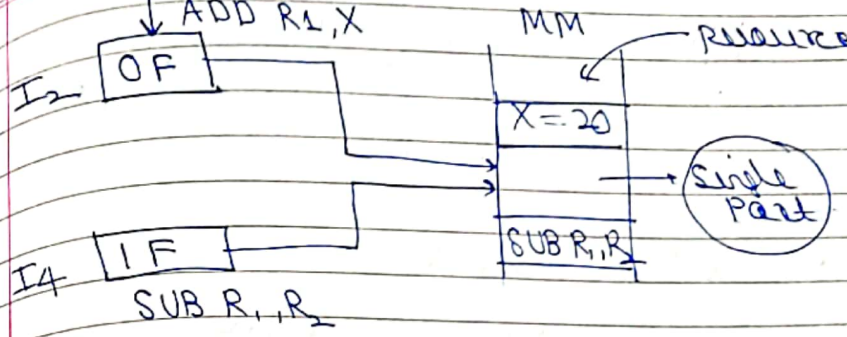
Strict Part: Only 1 unit can access mem.



: Resource duplication

4 CC

ADD R1, X → R1 ← R1 + M[X] (direct)



→ If 2/more Steps simultaneously request same resource leads to resource conflict.

- The solution can be :

(i) Use of dual/multi-ported memories [h/w implementation]

(ii) Use of separate instruction & data memories.

[1 module → OF ; another → IF]

(iii) Resource duplication (Default)

Providing multiple copies of Same resource.

→ With these H/W Solutions, no Stall cycles.

Scanned with CamScanner

* Instruction Formats and Addressing Modes

→ Basic Instr Format:

Opcode	Address Part

4 — 16 bit — 12

16 operations

→ Arithmetic
Branch
logical
Shift ...

2^{12} locations

→ three address instruction
⇒ ADD R_1, X, Y
$R_1 \leftarrow M[X] + M[Y]$

→ two address instruction
ADD R_1, X
$R_1 \leftarrow R_1 + M[X]$

→ one address
ADD X
$AC \leftarrow AC + M[X]$

→ zero address instruction
(Stack Instruction)
ADD
(both operands from top of Stack)

→ RISC instruction → manipulate data in registers only. [fast execution]
⇒ ADD R_1, R_2, R_3
$R_1 \leftarrow R_2 + R_3$
(imp)

→ type of operand.

→ The way the operand is chosen by the processor depends on Addressing Modes.

eg.

Opcode	Mode	Address part
4	3	9

16

(23 Modes)

(Pg 159)

ADD 10(R1) #300

Opcode	M1	A1	M2	A2
1011	010	Binary	101	Binary

CPU

translator

Scanned with CamScanner

* The Various Addressing Modes are :

1. 1. Register (000)
2. 2. Register indirect (001)
3. 3. Scaled Register indirect (010)
4. 4. Immediate (011)
5. 5. Direct/Absolute (100)
6. 6. Indirect (101)
7. 7. Zero Addressing (110)
8. 8. Displacement Addr. (111)

given

8 Displacement Addressing Mode :

EA = Address Part + Register

Address → offset

offset (displacement) → Address

→ 1 memory reference plus 1 ALU operation

→ Provides fast execution (than Indirect)

(ii) Relative Addressing Mode :

EA = offset + PC } Branch, control Statements.

399 : BNZ 50.

PC → 400 :

450

EA = 50 + PC = 450

(iii) Indexed Addressing Mode :

EA = Address + IR } Array indexing.

Base Address of Array.

Index register

(iii) Base Register Addressing Mode :

EA = offset + BR

(Base Add of Memory)

(relocatable code / Port^n indep. code) used in structures / records / subprograms / functions.

Q [3 Type of OS]

Q Match the following :

```
Struct student S1;
int x = 1;
A[i] = B[j];
while(A*++)
int *p = &y;
```

indexed
auto inc/dec
indirect
base register
immediate
displacement

Scanned with CamScanner

Consider a 2 word instruction:

ADD R1, @X, #Y

- R1 → Register
- @X → displacement
- #Y → indirect.

(i) No. of memory references for accessing data

$R_1 \leftarrow @X + \#Y$

① ②

∴ 3 memory references

(ii) No. of memory references for execution / completion

2 + ① + ② = 5.

2 word Instruction Fetch.

Q ~~ADD~~ @X, #Y.

(i) @X ← @X + #Y ∴ 4 Mem. references.

① ① ②

(ii) 2 + (1 + 1 + 2) = 6

Q ADD 10(R1), @X.

10(R1) ← 10(R1) + @X

① ① ①

∴ 3 mem references

Q 3 + 2 = 5.

Q 32 bit instruction
32 registers
20 operations
8 addressing modes
512 mem. Locations

→ 3 address instruction.

ADDI	R1	300	10

Reg. Add. Value.

→ 32 reg: 5 bits

→ ③

Opcode	M1	A1	M2	A2	M3	A3

→ x

20 op = 5 bits

8 modes = 3

③

512 loc = 9 bits

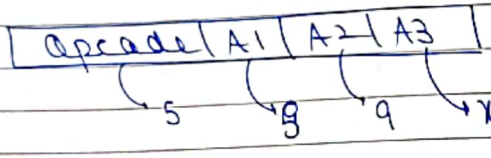
∴ x = 4 bits

Scanned with CamScanner

Unsigned : 0 to 15

Signed : -8 to +7 (2's comp.)

⇒ If Explicitly address modes not given :



x = 13 bits

UnSigned : → 0 to $2^{13} - 1$

Signed → -2^{12} to $2^{12} - 1$

* DATA REPRESENTATION

1. fixed point representation
2. ~~floating~~

- ~~generally~~ used for small range of values in the computer system
- integer data.

1. Signed Magnitude form
2. 1's complement form
3. 2's complement form

→ Representn
→ Range with K bits
→ overflow condition.

} 100% 1 Mark.

2. Floating point Representation

- Large range of values
- float or real data.

$(\pm m \times r^{\pm e})$ → exponent.
→ radix
→ mantissa

Eg: 32.625×10^4.

Eg: 1101.101×2^6 → exponent.
(mantissa)

→ Radix/Base is assumed.

① Normalized floating point number

→ 0.1101101×2^{10}
(Mantissa)

$\pm 0.1bbb\cdots b * 2^{\pm e}$

position of radix point is assumed.

MSB of Mantissa → 1.

Scanned with CamScanner

$\Rightarrow (-1) * (1.0111111) * 2^4$

$\Rightarrow -(10111.111)_2$

$\Downarrow$

$-(23.875)_{10}$ Ans

What is max value in Single precision?

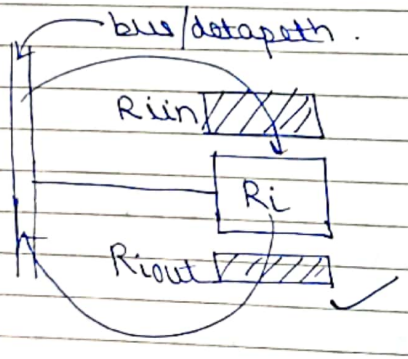
S = 0

E' = 254

M = 1111 ..... 1 (23)

* CONTROL UNIT DESIGN

- The purpose of Control Unit is to provide appropriate timing & control Signals.



if Riin = 1 $\Rightarrow$ Ri $\leftarrow$ bus

if Riout = 1 $\Rightarrow$ bus $\leftarrow$ Ri

eg: R4 $\leftarrow$ R3

R3out, R4in

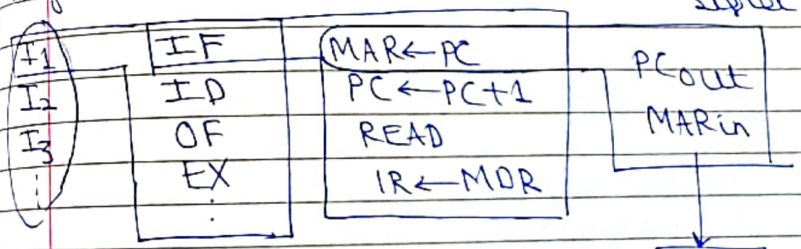
Control Signals

monitored by

CU

Everything designed at time of Processor design

Prog. Instcycle Microoperⁿ Control Signal



VI

CU

① Hardwired CU

② Microprog CU.

Scanned with CamScanner

4. 100CS ⟶ 100 SOP.

* HARDWIRED CONTROL UNIT DESIGN

- Control Functions are implemented in H/W.
- Every Cntrol Signal is represented in SOP expression and realized using digital hardware.

eg: PCin, MARout ⟶ Seperate SOP expression.

eg: $Y_{in} = T_1 + T_3 \cdot ADD + T_5 BR \cdots$

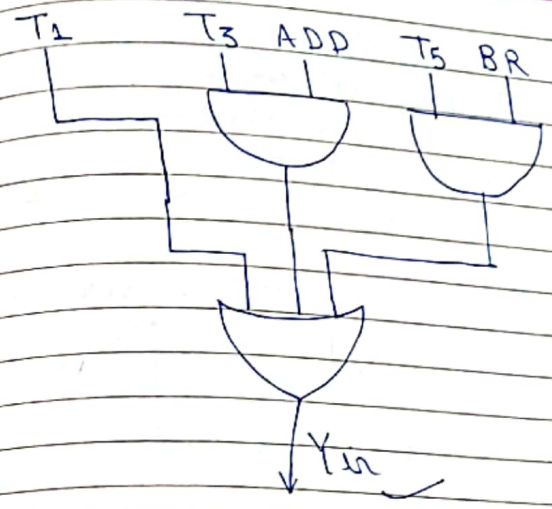
→ Relatively control signals are generated fast.
→ Less flexible to accomodate changes.
→ Everything is predefined.
→ For large number of instructions / control signals, complexity in h/w is more.
→ Implemented in RISC processor.

eg:

$Y_{in} = T_1 + T_3 \cdot ADD + T_5 \cdot BR + \cdots\cdots$

- Here, Yin gets enabled; during T_1 for all instructions, during T_3 for ADD and so on.

→ It can be realized as:

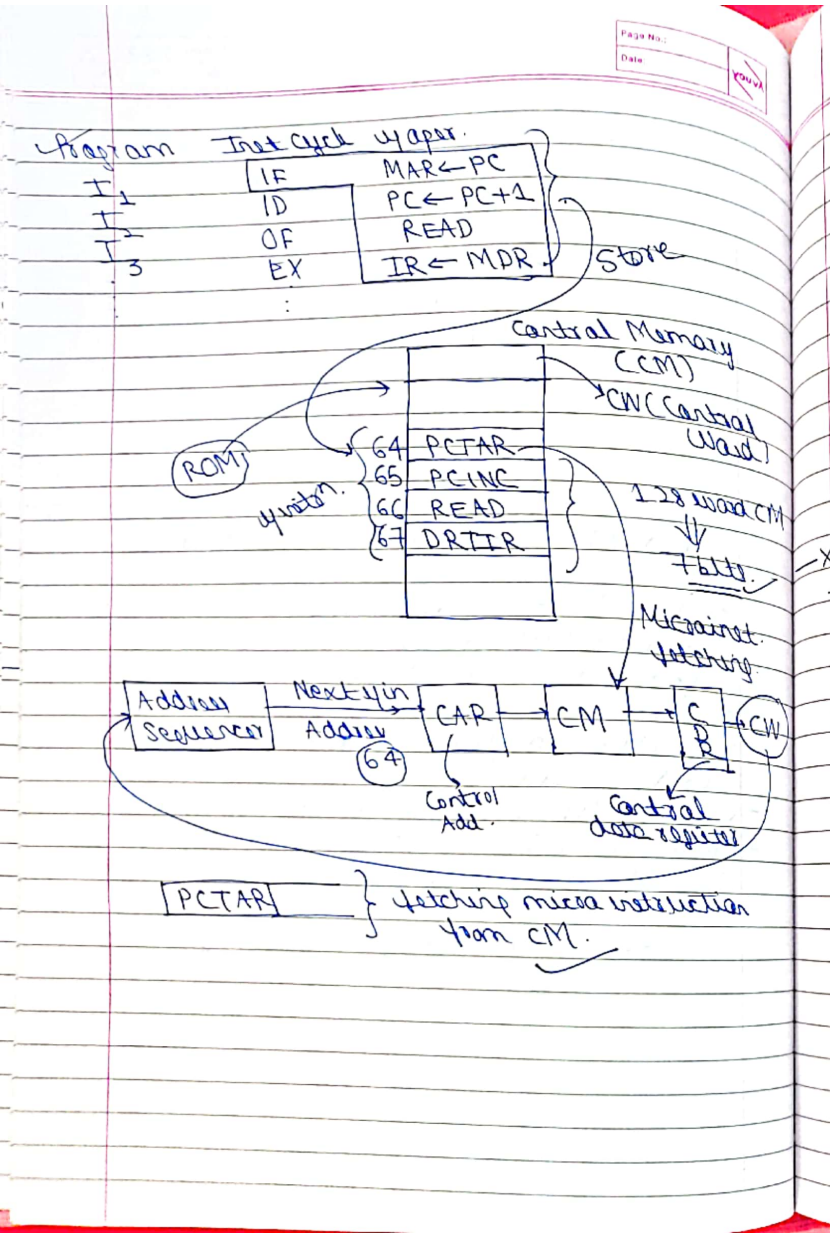


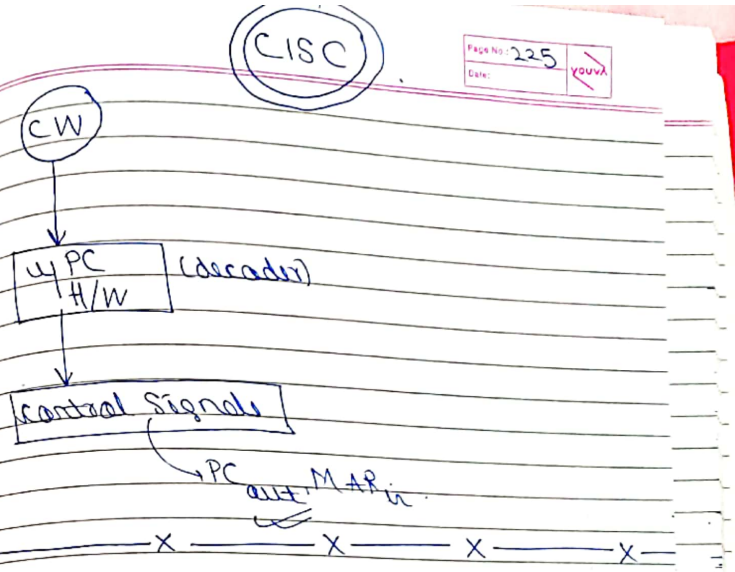
* Micro-Programmed Control Unit Design

- Control functions are implemented in S/W.
- Every control signal is generated after decoding micro-instruction.
- More flexible to accomodate changes.
- Effective for large no. of instructions & control signals.
- Relatively control signals are generated slower.
- Implemented in CISC Processor.
- Control Memory is ROM.

Storing ⟶ Fetching ⟶ decoding

Scanned with CamScanner





Scanned with CamScanner